Contents

1. Objective . . . 3

2. Reference Tools . . . 3

3. Approach . . . 3

4. Architecture . . . 4

4.1 Docker architecture . . . 4
4.2 Project files . . . 4
4.3 Containerized deep-learning application . . . 4
4.4 Running the project . . . 5
4.5 Container lifecycle . . . 5
4.6 Networking and storage layout . . . 6
4.7 Testing and issues resolved . . . 6

5. Source Code . . . 7

5.1 dl_app/app.py - containerized inference script . . . 7
5.2 dl_app/Dockerfile . . . 7
5.3 task9_docker.py - header, imports and build context . . . 7
5.4 task9_docker.py - step definitions (the 37 commands) . . . 8
5.5 task9_docker.py - command execution and logging . . . 10
5.6 task9_docker.py - PDF report generator and entry point . . . 11

6. Command Execution Log . . . 13

6.1 Install Docker and Verify the Setup (Steps 1-3) . . . 13
6.2 Create and Manage Containers (Steps 4-10) . . . 14
6.3 Explore Docker Images and Registries (Steps 11-17) . . . 16
6.4 Container Lifecycle Operations (Steps 18-26) . . . 18
6.5 Networking Concepts (Steps 27-31) . . . 19
6.6 Storage Concepts (Volumes and Bind Mounts) (Steps 32-36) . . . 20
6.7 Cleanup (Step 37) . . . 21

7. Screenshots and Observations . . . 21

7.1 Installation Check and hello-world (Steps 1-4) . . . 22
7.2 Container Creation, Access, Logs and Resource Usage (Steps 4-9) . . . 23
7.3 Docker Desktop - Containers Tab . . . 24
7.4 Failed Inspect, Image Pull, Registry Search, Image List and History (Steps 10-14) . . . 25
7.5 Layer History and Building the Custom Image (Steps 14-15) . . . 26
7.6 Docker Desktop - Images Tab . . . 27
7.7 Running the DL App, Tagging and Lifecycle from Create to Stop (Steps 15-22) . . . 28
7.8 Restart, Kill, Restart Policy, Removal and Networks (Steps 23-30) . . . 29
7.9 Network Inspection, Volumes, Bind Mount and Cleanup (Steps 30-37) . . . 30
7.10 Docker Desktop - Volumes Tab . . . 31

8. Results . . . 32

8.1 Results by approach area . . . 32
8.2 Lifecycle states observed (steps 18-26) . . . 32
8.3 Image inventory (steps 13 and 17, Figure 6) . . . 33
8.4 Network and storage objects . . . 33

9. Observations . . . 33

10. Known Limitations . . . 34

11. Conclusion . . . 34

1. Objective

To understand Docker architecture and manage containerized deep learning applications: install Docker, create and manage containers, explore images and registries, execute container lifecycle operations, and study networking and storage concepts, documenting every step with command logs, screenshots and observations.

2. Reference Tools

The task uses two reference tools. **Docker Desktop** (v4.94.0 on macOS) provides the engine, a Linux virtual machine and the graphical dashboard (Containers, Images, Volumes tabs). The **Docker CLI** (client 29.8.2, server 29.8.2) was used for all 37 recorded commands, driven by a Python script that stores the exact command, output, exit code and time of each step.

Item	Value from the recorded run
Docker version	29.8.2 (client and server)
Docker Desktop	v4.94.0, status 'Engine running'
Storage driver / OS	overlayfs / Docker Desktop (Linux VM)
Resources shown by Docker Desktop	10 CPUs available; memory limit 7.748 GiB (docker stats)
Image architecture	arm64 (hello-world 'arm64v8', Debian 'arm64' base layer)
Registry	Docker Hub (docker.io/library/...)

3. Approach

The five approach steps of the task were carried out in order. Section and figure numbers refer to this report.

1. **Install Docker** - Docker Desktop was installed and checked with `docker --version`, `docker version`, `docker info` and `docker run hello-world` (steps 1-3, Figure 1).
2. **Create and manage containers** - an nginx:alpine container named dl-web was started with port mapping and examined with ps, curl, exec, logs, stats and inspect (steps 4-10, Figures 2-3).
3. **Explore Docker images and registries** - python:3.11-slim was pulled, the registry was searched, images and layer history were listed, a custom image dl-app:1.0 was built from a Dockerfile, run and tagged (steps 11-17, Figures 4-6).
4. **Execute container lifecycle operations** - create, start, pause, unpause, stop, restart, kill, restart policy and remove, with the status shown after each command (steps 18-26, Figures 7-8).
5. **Study networking and storage concepts** - default networks, a user-defined bridge network with name resolution, a named volume with a persistence test, a bind mount and cleanup (steps 27-37, Figures 8-10).

4. Architecture

Docker follows a client-server design. The CLI and Docker Desktop's GUI send requests to the daemon, which builds and stores images, creates containers through containerd and runc, manages networks and volumes, and pulls images from a registry. On a Mac the engine runs inside a Linux VM, which is why paths such as /var/lib/docker/volumes appear inside that VM and not on macOS.

4.1 Docker architecture

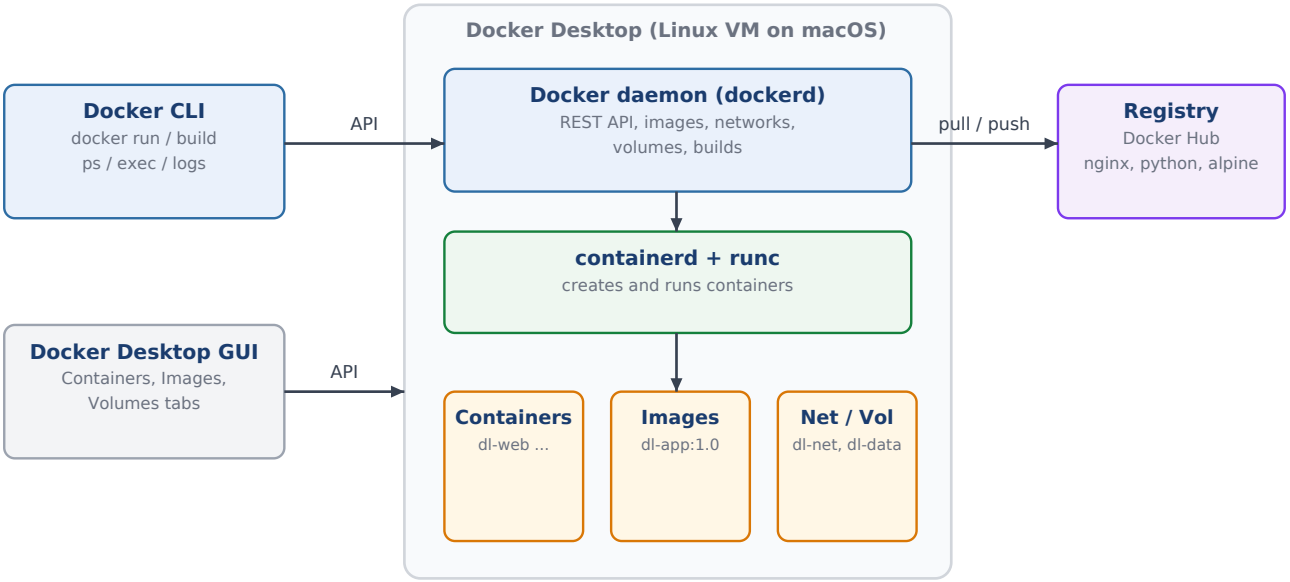


Diagram 1: Docker components used in this task: CLI, Docker Desktop GUI, daemon, containerd/runc and the registry.

4.2 Project files

File	Role
`task9_docker.py`	Runner: writes the build context, executes the 37 steps, saves the log and results, builds a first PDF.
`Task9_Docker.ipynb`	Notebook version of the same steps (one cell per step).
`dl_app/app.py`	Demonstration inference script run inside the container.
`dl_app/Dockerfile`	Recipe for the custom image dl-app:1.0.
`docker_command_log.txt`	Plain-text log of every command, its output and its exit code.
`docker_results.json`	Structured results of the run (command, output, exit code, time, section).
`host_data/sample.txt`	File created on the Mac for the bind-mount test.
`screenshots/`	The ten screenshots used in Section 7.

4.3 Containerized deep-learning application

The custom image packages a small neural-network forward pass written in NumPy. Four input samples with 8 features pass through a hidden layer with 16 ReLU units and an output layer with 3 softmax classes (8x16 + 16 + 16x3 + 3 = 195 parameters). The weights are random numbers from a fixed seed (42), so the app is a stand-in for an inference service that demonstrates packaging and running a deep-learning workload in a container; it is not a trained model.

The image is built from the following Dockerfile; the table lists what each instruction does.

Instruction	Effect (build stage in the log)
`FROM python:3.11-slim`	Base image with Python 3.11 on Debian (arm64) - stage [1/4]
`WORKDIR /app`	Creates and selects the working directory - stage [2/4]
`RUN pip install --no-cache-dir numpy`	Installs NumPy 2.4.6 into the image (7.6 s) - stage [3/4]
`COPY app.py .`	Adds the application script - stage [4/4]
`CMD ["python", "app.py"]`	Default command when a container starts (metadata only)

4.4 Running the project

- Install Docker Desktop, start it and wait until it shows *Engine running*.
- Put `task9_docker.py` in a folder, run `pip3 install reportlab` once, then `python3 task9_docker.py`. The run needs internet access to pull images from Docker Hub.
- The script creates `docker_command_log.txt`, `docker_results.json` and a first `Task9_Docker_Report.pdf`. Screenshots placed in `screenshots/` are embedded when the script is run with `--pdf-only`.
- Alternatively open `Task9_Docker.ipynb` and run all cells.

4.5 Container lifecycle

docker create makes a Created container; docker run = create + start (+ pull if the image is missing).
Exit code of the stopped container: 137 (= 128 + 9, SIGKILL) in steps 22 and 24.

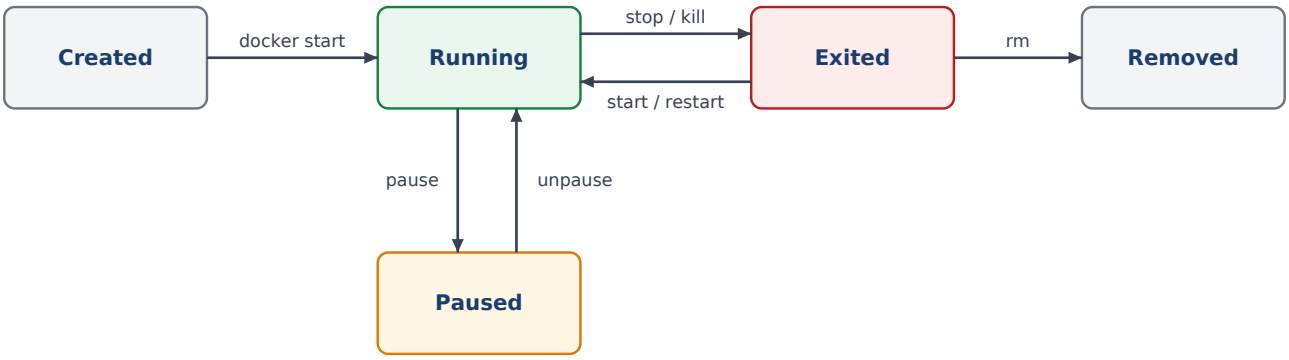


Diagram 2: Container states and the commands that cause each transition, as exercised in steps 18 to 26.

4.6 Networking and storage layout

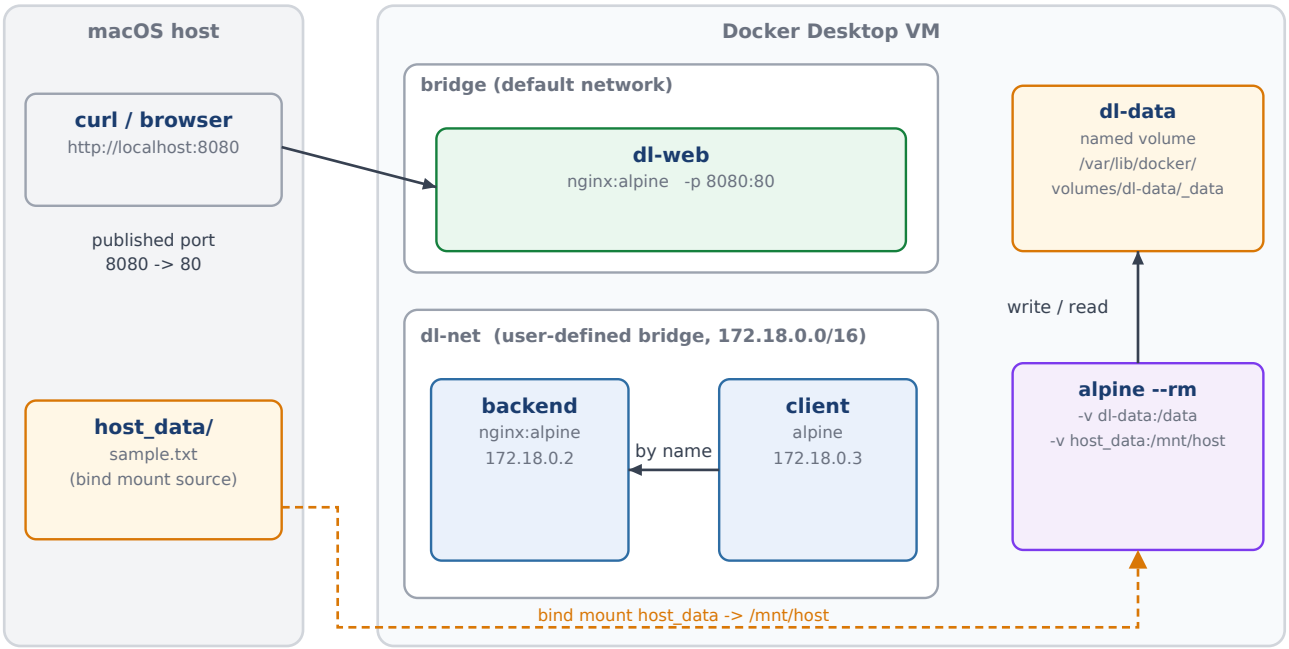


Diagram 3: Network and storage objects created in steps 4, 27 to 36: the published port, the user-defined network dl-net with its two containers and IP addresses, the named volume dl-data and the bind mount.

4.7 Testing and issues resolved

Issue found	Cause	Outcome
Step 10 failed with exit code 1	The template used `.NetworkSettings.IPAddress`. The error 'map has no entry for key IPAddress' shows that this key is absent from this engine's inspect output.	No later step depends on it. The container IPs were obtained in step 31 with `docker network inspect` (172.18.0.2 and 172.18.0.3). The per-network form `{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}` is the usual replacement; it was not re-run for this report.
Progress text labelled [stderr] in successful steps (4, 7, 8, 15, 29)	Docker writes pull and build progress, and nginx writes its version and log notices, to stderr.	All of these steps have exit code 0; no action needed.
Stop ended with Exited (137), not 0	sleep runs as PID 1 without a SIGTERM handler, so Docker sent SIGKILL after the 2-second timeout.	Expected behaviour; documented in step 22.
pip warnings during the image build	pip ran as root in the base image and a newer pip release exists.	Warnings only (build exit code 0). A production image would use a non-root user and pinned versions.
Docker Desktop screenshots need live resources	Step 37 deletes dl-web, dl-net and dl-data.	dl-web, dl-data and dl-net were re-created by hand after the run (visible at the end of Figure 9); images were kept.

5. Source Code

The container application, its Dockerfile and the runner script are listed in full. Lines longer than 100 characters are wrapped and continued with the symbol ↳. The notebook `Task9_Docker.ipynb` calls the same step definitions cell by cell and is submitted as a separate file.

5.1 dl_app/app.py - containerized inference script

```
 1 import numpy as np
 2
 3 # Tiny 2-layer neural network forward pass (stand-in for a DL inference service)
 4 rng = np.random.default_rng(42)
 5 x = rng.normal(size=(4, 8))                # batch of 4 samples, 8 features
 6 W1, b1 = rng.normal(size=(8, 16)), np.zeros(16)
 7 W2, b2 = rng.normal(size=(16, 3)), np.zeros(3)
 8
 9 h = np.maximum(0, x @ W1 + b1)            # ReLU hidden layer
10 logits = h @ W2 + b2
11 probs = np.exp(logits) / np.exp(logits).sum(axis=1, keepdims=True)
12
13 print("Containerized DL inference app")
14 print("NumPy version :", np.__version__)
15 print("Output shape  :", probs.shape)
16 print("Predicted classes:", probs.argmax(axis=1).tolist())
```

5.2 dl_app/Dockerfile

```
1 FROM python:3.11-slim
2 WORKDIR /app
3 RUN pip install --no-cache-dir numpy
4 COPY app.py .
5 CMD ["python", "app.py"]
```

5.3 task9_docker.py - header, imports and build context

```
 1 #!/usr/bin/env python3
 2 """
 3 Task 9: Docker Fundamentals and Container Lifecycle Management
 4 ---------------------------------------------------------------
 5 Runs every Docker step required by the task on YOUR machine, saves the real
 6 command log, and builds the PDF report (source code + outputs + observations).
 7
 8 Usage:
 9     1. Start Docker Desktop and wait until it says "Engine running".
10     2. python task9_docker.py            -> runs all steps, writes log + PDF
11        python task9_docker.py --pdf-only -> rebuild PDF from an existing log
12     3. (Optional) put Docker Desktop screenshots in ./screenshots/ BEFORE
13        building the PDF; they are appended automatically.
14
15 Outputs:
16     docker_command_log.txt   plain-text log of every command + output
17     docker_results.json      structured results (used for the PDF)
18     Task9_Docker_Report.pdf  report to upload on the L&T Edutech LMS
19 """
20 import glob
21 import json
22 import os
23 import subprocess
24 import sys
25 import textwrap
26 from datetime import datetime
27
28 HERE = os.path.dirname(os.path.abspath(__file__))
29 os.chdir(HERE)
30
31 LOG_TXT = "docker_command_log.txt"
32 LOG_JSON = "docker_results.json"
```

```
PDF_OUT = "Task9_Docker_Report.pdf"

# ---------------------------------------------------------------------------
# Files used by the custom image (a tiny "deep learning" inference app)
# ---------------------------------------------------------------------------
APP_PY = '''import numpy as np

# Tiny 2-layer neural network forward pass (stand-in for a DL inference service)
rng = np.random.default_rng(42)
x = rng.normal(size=(4, 8))                  # batch of 4 samples, 8 features
W1, b1 = rng.normal(size=(8, 16)), np.zeros(16)
W2, b2 = rng.normal(size=(16, 3)), np.zeros(3)

h = np.maximum(0, x @ W1 + b1)               # ReLU hidden layer
logits = h @ W2 + b2
probs = np.exp(logits) / np.exp(logits).sum(axis=1, keepdims=True)

print("Containerized DL inference app")
print("NumPy version :", np.__version__)
print("Output shape  :", probs.shape)
print("Predicted classes:", probs.argmax(axis=1).tolist())
'''

DOCKERFILE = """FROM python:3.11-slim
WORKDIR /app
RUN pip install --no-cache-dir numpy
COPY app.py .
CMD ["python", "app.py"]
"""


def write_build_context():
    os.makedirs("dl_app", exist_ok=True)
    with open("dl_app/app.py", "w") as f:
        f.write(APP_PY)
    with open("dl_app/Dockerfile", "w") as f:
        f.write(DOCKERFILE)
```

5.4 task9_docker.py - step definitions (the 37 commands)

```
# ---------------------------------------------------------------------------
# Steps: (section, title, command, observation)
# ---------------------------------------------------------------------------
S1 = "1. Install Docker and verify the setup"
S2 = "2. Create and manage containers"
S3 = "3. Explore Docker images and registries"
S4 = "4. Container lifecycle operations"
S5 = "5. Networking concepts"
S6 = "6. Storage concepts (volumes and bind mounts)"
S7 = "7. Cleanup"

STEPS = [
    # ---- 1 ----
    (S1, "Docker client/server version", "docker --version && docker version --format 'Client:
 ↳  {{.Client.Version}} | Server: {{.Server.Version}}'",
     "The client (CLI) talks to the Docker daemon (server) over a socket. Both versions confirm the
 ↳  installation works."),
    (S1, "Engine information", "docker info --format 'Containers: {{.Containers}} | Images:
 ↳  {{.Images}} | Storage driver: {{.Driver}} | OS: {{.OperatingSystem}}'",
     "Shows the daemon's state, the storage driver (overlay2 is typical) and the host OS the engine
 ↳  runs on."),
    (S1, "Run the hello-world test container", "docker run --rm hello-world",
     "The daemon did not have the image locally, so it pulled it from Docker Hub, created a
 ↳  container, ran it, and streamed the output back to the CLI."),

    # ---- 2 ----
    (S2, "Run a detached web container with port mapping", "docker run -d --name dl-web -p 8080:80
 ↳  nginx:alpine",
     "-d runs in the background, --name gives a friendly name, -p maps host port 8080 to container
 ↳  port 80."),
    (S2, "List running containers", "docker ps",
     "Shows container ID, image, command, status, port mapping and name of running containers."),
```

```
 97     (S2, "Call the container from the host", "curl -s -o /dev/null -w 'HTTP status: %{http_code}\\n'
    ↳     http://localhost:8080",
 98      "HTTP 200 proves the published port forwards host traffic into the container."),
 99     (S2, "Run a command inside the running container", "docker exec dl-web sh -c 'echo Hello from
    ↳     inside the container && nginx -v && hostname'",
100      "docker exec starts an extra process inside an existing container; the hostname equals the
    ↳     container ID."),
101     (S2, "View container logs", "docker logs --tail 5 dl-web",
102      "Logs capture the container's stdout/stderr (nginx access logs here)."),
103     (S2, "Resource usage snapshot", "docker stats --no-stream dl-web",
104      "Shows live CPU, memory, network and block I/O of the container."),
105     (S2, "Inspect container metadata", "docker inspect --format 'Name={{.Name}}
    ↳     State={{.State.Status}} IP={{.NetworkSettings.IPAddress}} Image={{.Config.Image}}' dl-web",
106      "docker inspect returns the full JSON configuration; --format extracts specific fields."),
107
108     # ---- 3 ----
109     (S3, "Pull an image from the registry (Docker Hub)", "docker pull python:3.11-slim",
110      "Images are made of read-only layers identified by digests; layers already present are not
    ↳     downloaded again."),
111     (S3, "Search the registry", "docker search tensorflow --limit 5",
112      "Docker Hub is the default public registry; search lists matching repositories with stars and
    ↳     official flags."),
113     (S3, "List local images", "docker images",
114      "Shows repository, tag, image ID, creation date and size of each local image."),
115     (S3, "Image layers (history)", "docker history python:3.11-slim",
116      "Every Dockerfile instruction creates a layer; this lists them with their sizes."),
117     (S3, "Build a custom DL-style image from a Dockerfile", "docker build -t dl-app:1.0 ./dl_app",
118      "docker build reads the Dockerfile, executes each instruction as a cached layer, and tags the
    ↳     result dl-app:1.0."),
119     (S3, "Run the custom image", "docker run --rm dl-app:1.0",
120      "Runs the containerized NumPy neural-network inference; the container exits when the process
    ↳     finishes."),
121     (S3, "Tag the image for a registry", "docker tag dl-app:1.0 yourdockerhubuser/dl-app:1.0 &&
    ↳     docker images yourdockerhubuser/dl-app",
122      "Tagging prepares the image to be pushed with 'docker push' to a registry (replace
    ↳     yourdockerhubuser with your account)."),
123
124     # ---- 4 ----
125     (S4, "Create a container without starting it", "docker create --name lifecycle-demo
    ↳     python:3.11-slim sleep 600 && docker ps -a --filter name=lifecycle-demo --format 'table
    ↳     {{.Names}}\\t{{.Status}}'",
126      "State: Created. The writable layer exists but no process is running."),
127     (S4, "Start", "docker start lifecycle-demo && docker ps --filter name=lifecycle-demo --format
    ↳     'table {{.Names}}\\t{{.Status}}'",
128      "State: Running."),
129     (S4, "Pause", "docker pause lifecycle-demo && docker ps --filter name=lifecycle-demo --format
    ↳     'table {{.Names}}\\t{{.Status}}'",
130      "State: Paused. Processes are frozen using the cgroup freezer (memory is kept)."),
131     (S4, "Unpause", "docker unpause lifecycle-demo && docker ps --filter name=lifecycle-demo
    ↳     --format 'table {{.Names}}\\t{{.Status}}'",
132      "Processes resume exactly where they stopped."),
133     (S4, "Stop (graceful, SIGTERM then SIGKILL)", "docker stop -t 2 lifecycle-demo && docker ps -a
    ↳     --filter name=lifecycle-demo --format 'table {{.Names}}\\t{{.Status}}'",
134      "State: Exited. Docker sends SIGTERM, waits, then SIGKILL; the filesystem is kept."),
135     (S4, "Restart", "docker restart lifecycle-demo && docker ps --filter name=lifecycle-demo
    ↳     --format 'table {{.Names}}\\t{{.Status}}'",
136      "Stop followed by start; the container returns to Running."),
137     (S4, "Kill (immediate SIGKILL)", "docker kill lifecycle-demo && docker ps -a --filter
    ↳     name=lifecycle-demo --format 'table {{.Names}}\\t{{.Status}}'",
138      "Forces immediate termination, exit code 137 (128 + 9)."),
139     (S4, "Restart policy demo", "docker run -d --name policy-demo --restart unless-stopped
    ↳     nginx:alpine && docker inspect --format 'RestartPolicy={{.HostConfig.RestartPolicy.Name}}'
    ↳     policy-demo",
140      "Restart policies (no, on-failure, always, unless-stopped) let the daemon restart containers
    ↳     automatically."),
141     (S4, "Remove containers", "docker rm -f lifecycle-demo policy-demo && docker ps -a --format
    ↳     'table {{.Names}}\\t{{.Status}}'",
142      "docker rm deletes the container and its writable layer; the image remains."),
143
144     # ---- 5 ----
145     (S5, "List default networks", "docker network ls",
146      "Default networks: bridge (default for containers), host and none."),
147     (S5, "Create a user-defined bridge network", "docker network create dl-net",
148      "User-defined bridges provide automatic DNS-based service discovery between containers."),
149     (S5, "Start two containers on the network", "docker run -d --name backend --network dl-net
    ↳     nginx:alpine && docker run -d --name client --network dl-net alpine sleep 300",
150      "Both containers join dl-net and get private IP addresses."),
151     (S5, "Container-to-container communication by name", "docker exec client sh -c 'ping -c 2
    ↳     backend && wget -qO- http://backend | head -n 4'",
152      "The name 'backend' resolves through Docker's embedded DNS, proving container-to-container
```

```
    ↳    networking."),
153     (S5, "Inspect the network", "docker network inspect dl-net --format '{{range
    ↳    .Containers}}{{.Name}} -> {{.IPv4Address}}{{println}}{{end}}'",
154      "Lists the containers attached to the network with their IP addresses."),
155
156     # ---- 6 ----
157     (S6, "Create a named volume", "docker volume create dl-data && docker volume ls --filter
    ↳    name=dl-data",
158      "Volumes are managed by Docker and live outside the container's writable layer."),
159     (S6, "Write data from one container", "docker run --rm -v dl-data:/data alpine sh -c \"echo
    ↳    'model checkpoint v1' > /data/checkpoint.txt && ls -l /data\"",
160      "The container writes into the volume and is then deleted."),
161     (S6, "Read the data from a NEW container (persistence)", "docker run --rm -v dl-data:/data
    ↳    alpine cat /data/checkpoint.txt",
162      "The data survived container deletion, proving volume persistence (key for datasets and trained
    ↳     models)."),
163     (S6, "Bind mount a host folder", "mkdir -p host_data && echo 'dataset sample from host' >
    ↳    host_data/sample.txt && docker run --rm -v \"$(pwd)/host_data:/mnt/host\" alpine cat
    ↳    /mnt/host/sample.txt",
164      "A bind mount maps a host directory into the container; changes are visible on both sides."),
165     (S6, "Inspect the volume", "docker volume inspect dl-data --format 'Name={{.Name}}
    ↳    Driver={{.Driver}} Mountpoint={{.Mountpoint}}'",
166      "Shows where Docker stores the volume on the host."),
167
168     # ---- 7 ----
169     (S7, "Remove all demo resources", "docker rm -f dl-web backend client 2>&1; docker network rm
    ↳    dl-net; docker volume rm dl-data; docker ps -a --format 'table {{.Names}}\\t{{.Status}}'",
170      "Cleaning up avoids leftover containers, networks and volumes."),
171 ]
172
173 SECTION_THEORY = {
174     S1: "Docker uses a client-server architecture: the Docker CLI (client) sends REST API calls to
    ↳    the Docker daemon (dockerd), which manages images, containers, networks and volumes and
    ↳    delegates container execution to containerd and runc. Docker Desktop bundles the engine, CLI, a
    ↳    Linux VM (on Windows/macOS), and a GUI.",
175     S2: "A container is a running instance of an image: an isolated process using Linux namespaces
    ↳    (isolation) and cgroups (resource limits), with a thin writable layer on top of read-only image
    ↳    layers.",
176     S3: "An image is an immutable, layered template. A registry (Docker Hub, GHCR, ECR, private
    ↳    registries) stores and distributes images by repository:tag and digest. A Dockerfile describes
    ↳    how to build an image reproducibly, which is critical for deep learning environments with pinned
    ↳     library versions.",
177     S4: "Container states: Created -> Running -> Paused -> Exited -> Removed. Transitions are driven
    ↳      by create, start, pause/unpause, stop/kill, restart and rm.",
178     S5: "Docker network drivers: bridge (default, single host), host (shares host network stack),
    ↳    none (no networking), overlay (multi-host swarm). User-defined bridge networks add DNS
    ↳    resolution by container name and better isolation.",
179     S6: "Container filesystems are ephemeral. Volumes (Docker-managed) and bind mounts (host paths)
    ↳    persist data beyond the container lifecycle. tmpfs mounts keep data in memory only.",
180     S7: "",
181 }
```

5.5 task9_docker.py - command execution and logging

```
184 # ------------------------------------------------------------------------------
185 # Execution
186 # ------------------------------------------------------------------------------
187 def run_cmd(cmd, timeout=900):
188     """Run a shell command and return a result dict with REAL output."""
189     start = datetime.now()
190     try:
191         import shutil
192         bash = shutil.which("bash")  # Git Bash / WSL on Windows; /bin/bash on Linux/macOS
193         p = subprocess.run(cmd, shell=True, capture_output=True, text=True, timeout=timeout,
194                            executable=bash if (bash and os.name != "nt") else None) if os.name !=
    ↳    "nt" or not bash \
195             else subprocess.run([bash, "-c", cmd], capture_output=True, text=True, timeout=timeout)
196         out = (p.stdout or "") + (("\n[stderr]\n" + p.stderr) if p.stderr.strip() else "")
197         rc = p.returncode
198     except subprocess.TimeoutExpired:
```

```
        out, rc = "Command timed out.", 124
    return {"command": cmd, "output": out.rstrip(), "returncode": rc,
            "time": start.strftime("%Y-%m-%d %H:%M:%S")}


def run_step(i, results=None):
    section, title, cmd, obs = STEPS[i]
    write_build_context()
    r = run_cmd(cmd)
    r.update({"section": section, "title": title, "observation": obs})
    print(f"\n### [{i + 1}/{len(STEPS)}] {title}\n$ {cmd}\n{r['output']}\n(exit code
    ↳  {r['returncode']})")
    if results is not None:
        results.append(r)
    return r


def docker_ready():
    r = run_cmd("docker info", timeout=60)
    return r["returncode"] == 0


def save_log(results):
    with open(LOG_JSON, "w") as f:
        json.dump(results, f, indent=2)
    with open(LOG_TXT, "w") as f:
        f.write(f"Docker command log - generated {datetime.now():%Y-%m-%d %H:%M:%S}\n")
        f.write("=" * 78 + "\n")
        for i, r in enumerate(results, 1):
            f.write(f"\n[{i}] {r['section']} - {r['title']}  ({r['time']})\n")
            f.write(f"$ {r['command']}\n{r['output']}\n(exit code {r['returncode']})\n")
            f.write("-" * 78 + "\n")
```

5.6 task9_docker.py - PDF report generator and entry point

```
# ---------------------------------------------------------------------------
# PDF report
# ---------------------------------------------------------------------------
def build_pdf(results):
    from reportlab.lib import colors
    from reportlab.lib.pagesizes import A4
    from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
    from reportlab.lib.units import cm
    from reportlab.platypus import (Image, KeepTogether, PageBreak, Paragraph,
                                    Preformatted, SimpleDocTemplate, Spacer, Table, TableStyle)
    from xml.sax.saxutils import escape

    ss = getSampleStyleSheet()
    H1 = ParagraphStyle("H1", parent=ss["Heading1"], textColor=colors.HexColor("#1F3A6E"))
    H2 = ParagraphStyle("H2", parent=ss["Heading2"], textColor=colors.HexColor("#1F3A6E"))
    H3 = ParagraphStyle("H3", parent=ss["Heading3"], spaceBefore=8, spaceAfter=3)
    BODY = ParagraphStyle("B", parent=ss["BodyText"], leading=14)
    CMD = ParagraphStyle("CMD", fontName="Courier-Bold", fontSize=8, leading=10,
    ↳  backColor=colors.HexColor("#E8EEF7"),
                         borderPadding=4, leftIndent=2)
    OUT = ParagraphStyle("OUT", fontName="Courier", fontSize=7.2, leading=9,
    ↳  backColor=colors.HexColor("#F4F4F4"),
                         borderPadding=4, leftIndent=2)

    def wrap(text, width=105, max_lines=45):
        lines = []
        for ln in text.splitlines():
            lines.extend(textwrap.wrap(ln, width, replace_whitespace=False, drop_whitespace=False)
    ↳  or [""])
        if len(lines) > max_lines:
            lines = lines[:max_lines] + [f"... ({len(lines) - max_lines} more lines - see
    ↳  docker_command_log.txt)"]
```

```
        return "\n".join(lines)

    doc = SimpleDocTemplate(PDF_OUT, pagesize=A4, leftMargin=1.8 * cm, rightMargin=1.8 * cm,
                            topMargin=1.8 * cm, bottomMargin=1.8 * cm,
                            title="Task 9: Docker Fundamentals and Container Lifecycle Management")
    s = []
    s.append(Paragraph("Task 9: Docker Fundamentals and Container Lifecycle Management", H1))
    s.append(Paragraph("Practical Implementation Report", H2))
    s.append(Paragraph(f"Generated: {datetime.now():%d %B %Y, %H:%M}", BODY))
    s.append(Spacer(1, 10))

    s.append(Paragraph("1. Objective", H2))
    s.append(Paragraph("To understand Docker architecture and manage containerized deep learning applications.", BODY))
    s.append(Paragraph("2. Reference Tools", H2))
    s.append(Paragraph("Docker Desktop and Docker CLI.", BODY))

    # Environment
    env = next((r for r in results if r["title"].startswith("Docker client")), None)
    if env:
        s.append(Paragraph("3. Environment", H2))
        s.append(Preformatted(wrap(env["output"]), OUT))

    s.append(Paragraph("4. Summary of work (mapped to the required approach)", H2))
    ok = sum(1 for r in results if r["returncode"] == 0)
    rows = [["Required approach", "Steps", "Succeeded"]]
    for sec in [S1, S2, S3, S4, S5, S6]:
        rs = [r for r in results if r["section"] == sec]
        rows.append([sec, str(len(rs)), f"{sum(1 for r in rs if r['returncode'] == 0)}/{len(rs)}"])
    t = Table(rows, colWidths=[11 * cm, 2.2 * cm, 3 * cm])
    t.setStyle(TableStyle([("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#1F3A6E")),
                           ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
                           ("GRID", (0, 0), (-1, -1), 0.4, colors.grey),
                           ("FONTSIZE", (0, 0), (-1, -1), 9)]))
    s.append(t)
    s.append(Paragraph(f"Total: {ok}/{len(results)} commands completed with exit code 0.", BODY))

    s.append(Paragraph("5. Source code", H2))
    s.append(Paragraph("<b>dl_app/app.py</b>", BODY))
    s.append(Preformatted(APP_PY, OUT))
    s.append(Paragraph("<b>dl_app/Dockerfile</b>", BODY))
    s.append(Preformatted(DOCKERFILE, OUT))

    s.append(PageBreak())
    s.append(Paragraph("6. Command execution, outputs and observations", H1))
    cur, n = None, 0
    for r in results:
        if r["section"] != cur:
            cur = r["section"]
            s.append(Paragraph(cur, H2))
            if SECTION_THEORY.get(cur):
                s.append(Paragraph(f"<b>Concept:</b> {escape(SECTION_THEORY[cur])}", BODY))
        n += 1
        block = [Paragraph(f"Step {n}: {escape(r['title'])}", H3),
                 Preformatted(wrap("$ " + r["command"], 105, 12), CMD),
                 Spacer(1, 3),
                 Preformatted(wrap(r["output"] or "(no output)") + f"\n[exit code {r['returncode']}]", OUT),
                 Spacer(1, 3),
                 Paragraph(f"<b>Observation:</b> {escape(r['observation'])}", BODY)]
        s.append(KeepTogether(block[:4]))
        s.extend(block[4:])

    shots = sorted(glob.glob("screenshots/*.png") + glob.glob("screenshots/*.jpg") + glob.glob("screenshots/*.jpeg"))
    s.append(PageBreak())
    s.append(Paragraph("7. Screenshots (Docker Desktop)", H1))
    if shots:
        for p in shots:
            s.append(Paragraph(escape(os.path.basename(p)), H3))
            img = Image(p)
            max_w, max_h = 16.5 * cm, 20 * cm
            ratio = min(max_w / img.imageWidth, max_h / img.imageHeight, 1)
            img.drawWidth, img.drawHeight = img.imageWidth * ratio, img.imageHeight * ratio
            s.append(img)
            s.append(Spacer(1, 8))
    else:
        s.append(Paragraph("No screenshots found. Add PNG/JPG files to the 'screenshots' folder and re-run "
                           "<font face='Courier'>python task9_docker.py --pdf-only</font>.", BODY))
```

```
336
337     s.append(Paragraph("8. Overall observations and conclusion", H1))
338     for b in [
339         "Docker's client-server design lets the same CLI command work against any engine; the daemon
    ↳     performs all work.",
340         "Images are layered and cached, which made repeated pulls/builds fast and gave a
    ↳    reproducible environment for the DL app.",
341         "Containers are lightweight, isolated processes; the full lifecycle (create, start, pause,
    ↳    stop, kill, restart, remove) was exercised and verified through status output.",
342         "User-defined bridge networks provide name-based discovery; published ports (-p) expose
    ↳    services to the host.",
343         "Container storage is ephemeral; named volumes and bind mounts persist datasets and model
    ↳    checkpoints independently of container lifetime.",
344         "Taken together, these capabilities make Docker well suited for packaging, shipping and
    ↳    running deep learning workloads consistently across machines.",
345     ]:
346         s.append(Paragraph("&bull; " + escape(b), BODY))
347     doc.build(s)
348     print(f"\nPDF written: {PDF_OUT}")
349
350
351 def main():
352     if "--pdf-only" in sys.argv:
353         with open(LOG_JSON) as f:
354             build_pdf(json.load(f))
355         return
356     write_build_context()
357     if not docker_ready():
358         sys.exit("Docker is not running or not installed. Start Docker Desktop, wait for 'Engine
    ↳    running', "
359                  "then run this script again.")
360     results = []
361     for i in range(len(STEPS)):
362         run_step(i, results)
363     save_log(results)
364     build_pdf(results)
365     print(f"Log saved: {LOG_TXT}")
366
367
368 if __name__ == "__main__":
369     main()
```

6. Command Execution Log

All 37 commands below were run by `task9_docker.py`. The command, output and exit code are copied from `docker_results.json`; the observation under each step explains what the output shows. Very long pull and build outputs are shortened here (marked with '...'), and the complete text is in `docker_command_log.txt` and in the screenshots of Section 7.

6.1 Install Docker and Verify the Setup (Steps 1-3)

Concept: Docker uses a client-server architecture: the docker CLI sends API requests to the Docker daemon (dockerd), which manages images, containers, networks and volumes and hands container execution to containerd and runc. On macOS, Docker Desktop runs the engine in a lightweight Linux VM and adds a graphical dashboard.

Step 1: Docker client/server version (exit code 0)

```
$ docker --version && docker version --format 'Client: {{.Client.Version}} | Server: {{.Server.Version}}'
```

```
Docker version 29.8.2, build 7fc2dff
Client: 29.8.2 | Server: 29.8.2
```

Observation: Client and server both report version 29.8.2, so the CLI is talking to the Docker Desktop engine correctly.

Step 2: Engine information (exit code 0)

```
$ docker info --format 'Containers: {{.Containers}} | Images: {{.Images}} | Storage driver: {{.Driver}} | OS: {{.OperatingSystem}}'
```

```
Containers: 0 | Images: 1 | Storage driver: overlayfs | OS: Docker Desktop
```

Observation: The engine had 0 containers and 1 image before the run. The storage driver is overlayfs and the OS is reported as Docker Desktop (the engine runs inside Docker Desktop's Linux VM).

Step 3: Run the hello-world test container (exit code 0)

```
$ docker run --rm hello-world
```

```
Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (arm64v8)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Observation: hello-world was already present locally (no pull message), so Docker created a container, ran it, printed the four-step explanation and exited with code 0. --rm removed the container afterwards.

6.2 Create and Manage Containers (Steps 4-10)

Concept: A container is a running instance of an image: an isolated process (Linux namespaces) with resource limits (cgroups) and a thin writable layer on top of the image's read-only layers. A port is published to the host with -p host_port:container_port.

Step 4: Run a detached web container with port mapping (exit code 0)

```
$ docker run -d --name dl-web -p 8080:80 nginx:alpine
```

```
562445d82744770349bda1d3365204f0c3badd112b3cce1656a529d2d6b3c68e

[stderr]
Unable to find image 'nginx:alpine' locally
alpine: Pulling from library/nginx
1efec2485f45: Pulling fs layer
... [24 lines omitted - full output in docker_command_log.txt]
1efec2485f45: Pull complete
Digest: sha256:df221db836e1754089190208cee7eeda94f233197056426eda74a43ab1abeac2
Status: Downloaded newer image for nginx:alpine
```

Observation: nginx:alpine was not available locally, so the daemon pulled its 8 layers (digest sha256:df221db8...) before starting the container in the background. The printed value is the full container ID; the log shows it starts with 562445d82744. Pull progress is written to stderr, which is why it is labelled [stderr]; the exit code is 0.

Step 5: List running containers (exit code 0)

```
$ docker ps
```

```
CONTAINER ID   IMAGE          COMMAND                  CREATED                  STATUS                  PORTS
                                        NAMES
562445d82744   nginx:alpine   "/docker-entrypoint.…"   Less than a second ago   Up Less than a second
  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   dl-web
```

Observation: Exactly one container is running: dl-web from nginx:alpine, with host port 8080 mapped to container port 80 for both IPv4 (0.0.0.0) and IPv6 ([::]).

Step 6: Call the container from the host (exit code 0)

```
$ curl -s -o /dev/null -w 'HTTP status: %{http_code}\n' http://localhost:8080
```

```
HTTP status: 200
```

Observation: HTTP status 200 proves that the published port forwards traffic from the Mac into the container.

Step 7: Run a command inside the running container (exit code 0)

```
$ docker exec dl-web sh -c 'echo Hello from inside the container && nginx -v && hostname'
```

```
Hello from inside the container
562445d82744

[stderr]
nginx version: nginx/1.31.6
```

Observation: The hostname inside the container (562445d82744) equals the first 12 characters of the container ID. nginx -v prints its version (nginx/1.31.6) on stderr.

Step 8: View container logs (exit code 0)

```
$ docker logs --tail 5 dl-web
```

```
192.168.65.1 - - [06/Oct/2026:15:48:29 +0000] "GET / HTTP/1.1" 200 896 "-" "curl/8.16.0" "-"

[stderr]
2026/10/06 15:48:29 [notice] 1#1: start worker process 36
2026/10/06 15:48:29 [notice] 1#1: start worker process 37
2026/10/06 15:48:29 [notice] 1#1: start worker process 38
2026/10/06 15:48:29 [notice] 1#1: start worker process 39
```

Observation: The log holds one access-log line (GET / answered with 200, user agent curl/8.16.0), which is the request from step 6, plus four nginx start-up notices for worker processes 36 to 39. The client address 192.168.65.1 belongs to Docker Desktop's internal VM network rather than 127.0.0.1.

Step 9: Resource usage snapshot (exit code 0)

```
$ docker stats --no-stream dl-web
```

```
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT     MEM %     NET I/O          BLOCK I/O     PIDS
562445d82744   dl-web    0.00%     10.25MiB / 7.748GiB   0.13%     1.63kB / 1.6kB   0B / 8.19kB   11
```

Observation: The idle web server uses 0.00 % CPU and 10.25 MiB of 7.748 GiB memory (0.13 %), with 1.63 kB received / 1.6 kB sent and 11 processes.

Step 10: Inspect container metadata (exit code 1)

```
$ docker inspect --format 'Name={{.Name}} State={{.State.Status}} IP={{.NetworkSettings.IPAddress}}
  Image={{.Config.Image}}' dl-web
```

```
[stderr]
template parsing error: template: :1:60: executing "" at <.NetworkSettings.IPAddress>: map has no entry for key
  "IPAddress"
```

Observation (failed step): This is the only failed command (exit code 1). The Go template asked for .NetworkSettings.IPAddress, and the error 'map has no entry for key IPAddress' shows that this key does not exist in the engine's inspect output. It did not affect later steps; the container IPs were obtained successfully in step 31 (see Section 4.7).

6.3 Explore Docker Images and Registries (Steps 11-17)

Concept: An image is an immutable, layered template identified by repository:tag and a content digest. A registry such as Docker Hub stores and distributes images. A Dockerfile describes how to build an image reproducibly, which matters for deep learning environments with fixed library versions.

Step 11: Pull an image from the registry (Docker Hub) (exit code 0)

```
$ docker pull python:3.11-slim
```

```
3.11-slim: Pulling from library/python
7ca0c331ecfd: Pulling fs layer
9eb0fb93ee08: Pulling fs layer
bbeda6b4abb7: Pulling fs layer
73a63ac3ec4a: Pulling fs layer
9eb0fb93ee08: Download complete
7ca0c331ecfd: Download complete
58b126dd1754: Download complete
56f7c999317d: Download complete
bbeda6b4abb7: Download complete
73a63ac3ec4a: Download complete
bbeda6b4abb7: Pull complete
9eb0fb93ee08: Pull complete
7ca0c331ecfd: Pull complete
73a63ac3ec4a: Pull complete
Digest: sha256:0dd364ba7e10242f07755449e3a3d0e35f9efd987952737b90def6709ab0c5ce
Status: Downloaded newer image for python:3.11-slim
docker.io/library/python:3.11-slim
```

Observation: python:3.11-slim was downloaded as 4 layers (digest sha256:0dd364ba...). Docker reports 'Downloaded newer image', meaning the local copy was new.

Step 12: Search the registry (exit code 0)

```
$ docker search tensorflow --limit 5
```

```
NAME                    DESCRIPTION                                     STARS     OFFICIAL
tensorflow/tensorflow   Official Docker images for the machine learn…   2932
rocm/tensorflow         Tensorflow with ROCm backend support            86
photoprism/tensorflow   TensorFlow for C Build Environment              0
corpusops/tensorflow    https://github.com/corpusops/docker-tensorfl…   0
rootpublic/tensorflow                                                   0
```

Observation: tensorflow/tensorflow is the most popular match (2,932 stars), followed by rocm/tensorflow (86). The OFFICIAL column is empty for all results, so none is a Docker Official Image.

Step 13: List local images (exit code 0)

```
$ docker images
```

```
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest   5e2309035332       17.1kB         4.77kB
nginx:alpine         df221db836e1       93.9MB         27.1MB   U
python:3.11-slim     0dd364ba7e10        215MB         48.1MB
```

Observation: Three images were present at this point: hello-world (17.1 kB), nginx:alpine (93.9 MB disk usage, 27.1 MB content) and python:3.11-slim (215 MB, 48.1 MB). The U in the EXTRA column appears only for nginx:alpine, the image used by the running container.

Step 14: Image layers (history) (exit code 0)

```
$ docker history python:3.11-slim
```

```
IMAGE          CREATED        CREATED BY                                      SIZE      COMMENT
0dd364ba7e10   14 hours ago   CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      14 hours ago   RUN /bin/sh -c set -eux;  for src in idle3 p…   16.4kB    buildkit.dockerfile.v0
<missing>      14 hours ago   RUN /bin/sh -c set -eux;   savedAptMark="$(a…   52.1MB    buildkit.dockerfile.v0
<missing>      14 hours ago   ENV PYTHON_SHA256=bfb74ad39efae27cda510f134a…   0B        buildkit.dockerfile.v0
<missing>      14 hours ago   ENV PYTHON_VERSION=3.11.17                      0B        buildkit.dockerfile.v0
<missing>      14 hours ago   ENV GPG_KEY=A035C8C19219BA821ECEA86B64E628F8…   0B        buildkit.dockerfile.v0
<missing>      14 hours ago   RUN /bin/sh -c set -eux;  apt-get update;  a…   4.99MB    buildkit.dockerfile.v0
<missing>      14 hours ago   ENV LANG=C.UTF-8                                0B        buildkit.dockerfile.v0
<missing>      14 hours ago   ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buildkit.dockerfile.v0
<missing>      40 hours ago   # debian.sh --arch 'arm64' out/ 'trixie' '@1…   110MB     debuerreotype 0.17
```

Observation: The history lists the layers of python:3.11-slim from CMD ["python3"] down to the 110 MB Debian (arm64) base layer. Instructions that only set metadata (ENV, CMD) have size 0 B, while the two RUN layers add 52.1 MB and 4.99 MB. The <missing> IDs are normal for pulled images.

Step 15: Build a custom DL-style image from a Dockerfile (exit code 0)

```
$ docker build -t dl-app:1.0 ./dl_app
```

```
[stderr]
#0 building with "desktop-linux" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 148B done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/python:3.11-slim
... [47 lines omitted - full output in docker_command_log.txt]
#9 exporting layers 2.4s done
#9 exporting manifest sha256:f199e40ff0715e54c75d43014eaeca496644a3e7d68c7754b8c76824884de3d5 done
#9 exporting config sha256:22c9baabe670a037fced1dbf13b8b48ea2675e323c2ef9e04d86268989e73f6d done
#9 exporting attestation manifest sha256:27bea9899e1c00ca8993bf970a9a47a3c07deecbc5275774f44bc81e31312600 done
#9 exporting manifest list sha256:34d8650d4c166ed015da916d2feeb849735741166386ba77f644b8c7dec5489d done
#9 naming to docker.io/library/dl-app:1.0 done
#9 unpacking to docker.io/library/dl-app:1.0
#9 unpacking to docker.io/library/dl-app:1.0 0.5s done
#9 DONE 2.9s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/lx36dzav3rthmuxqjcs3mxmrp
```

Observation: The four build steps ran in order: FROM, WORKDIR /app, RUN pip install numpy (7.6 s, NumPy 2.4.6 aarch64 wheel of 16.0 MB) and COPY app.py. The image was exported and named docker.io/library/dl-app:1.0 in 2.9 s. BuildKit prints its progress on stderr, and the pip 'running as root' and 'new pip release' lines are warnings, not errors.

Step 16: Run the custom image (exit code 0)

```
$ docker run --rm dl-app:1.0
```

```
Containerized DL inference app
NumPy version : 2.4.6
Output shape  : (4, 3)
Predicted classes: [2, 0, 0, 0]
```

Observation: The container executed the NumPy inference script: NumPy 2.4.6, output shape (4, 3) and predicted classes [2, 0, 0, 0]. It exited automatically and --rm deleted it.

Step 17: Tag the image for a registry (exit code 0)

```
$ docker tag dl-app:1.0 yourdockerhubuser/dl-app:1.0 && docker images yourdockerhubuser/dl-app
```

```
IMAGE                          ID             DISK USAGE   CONTENT SIZE   EXTRA
yourdockerhubuser/dl-app:1.0   34d8650d4c16        324MB         70.7MB
```

Observation: docker tag only adds a second name for the same image: both names share image ID 34d8650d4c16 (324 MB disk usage, 70.7 MB content), so no extra storage is used. No push was made because the name is a placeholder.

6.4 Container Lifecycle Operations (Steps 18-26)

Concept: A container moves through the states Created, Running, Paused, Exited and Removed. The commands create, start, pause, unpause, stop, kill, restart and rm drive these transitions, and a restart policy lets the daemon restart containers automatically.

Step 18: Create a container without starting it (exit code 0)

```
$ docker create --name lifecycle-demo python:3.11-slim sleep 600 && docker ps -a --filter name=lifecycle-demo
  --format 'table {{.Names}}\t{{.Status}}'
```

```
3817f42343b9146ec3087eb9de5ca0e848c11f3008808ccb3c2364487f8730b2
NAMES            STATUS
lifecycle-demo   Created
```

Observation: State Created: the container exists with its writable layer, but no process is running.

Step 19: Start (exit code 0)

```
$ docker start lifecycle-demo && docker ps --filter name=lifecycle-demo --format 'table {{.Names}}\t{{.Status}}'
```

```
lifecycle-demo
NAMES            STATUS
lifecycle-demo   Up Less than a second
```

Observation: State Running (Up).

Step 20: Pause (exit code 0)

```
$ docker pause lifecycle-demo && docker ps --filter name=lifecycle-demo --format 'table {{.Names}}\t{{.Status}}'
```

```
lifecycle-demo
NAMES            STATUS
lifecycle-demo   Up Less than a second (Paused)
```

Observation: State Up ... (Paused): all processes are frozen but the container is still listed as up.

Step 21: Unpause (exit code 0)

```
$ docker unpause lifecycle-demo && docker ps --filter name=lifecycle-demo --format 'table
  {{.Names}}\t{{.Status}}'
```

```
lifecycle-demo
NAMES            STATUS
lifecycle-demo   Up Less than a second
```

Observation: The container is running again after unpause.

Step 22: Stop (graceful, SIGTERM then SIGKILL) (exit code 0)

```
$ docker stop -t 2 lifecycle-demo && docker ps -a --filter name=lifecycle-demo --format 'table
  {{.Names}}\t{{.Status}}'
```

```
lifecycle-demo
NAMES            STATUS
lifecycle-demo   Exited (137) Less than a second ago
```

Observation: docker stop -t 2 ended in Exited (137). The sleep process runs as PID 1 and has no SIGTERM handler, so Docker killed it after the 2-second grace period; 137 = 128 + 9 (SIGKILL).

Step 23: Restart (exit code 0)

```
$ docker restart lifecycle-demo && docker ps --filter name=lifecycle-demo --format 'table
  {{.Names}}\t{{.Status}}'
```

```
lifecycle-demo
NAMES            STATUS
lifecycle-demo   Up Less than a second
```

Observation: The restart returned the container to Running (Up Less than a second).

Step 24: Kill (immediate SIGKILL) (exit code 0)

```
$ docker kill lifecycle-demo && docker ps -a --filter name=lifecycle-demo --format 'table
  {{.Names}}\t{{.Status}}'
```

```
lifecycle-demo
NAMES            STATUS
lifecycle-demo   Exited (137) Less than a second ago
```

Observation: docker kill sends SIGKILL immediately; the container ends with Exited (137).

Step 25: Restart policy demo (exit code 0)

```
$ docker run -d --name policy-demo --restart unless-stopped nginx:alpine && docker inspect --format
  'RestartPolicy={{.HostConfig.RestartPolicy.Name}}' policy-demo
```

```
9820d01540a0ea5504cc38c63862186f68536cbdae4bfcb95d41df3f64ad575b
RestartPolicy=unless-stopped
```

Observation: A new nginx container was created with --restart unless-stopped and docker inspect confirms RestartPolicy=unless-stopped.

Step 26: Remove containers (exit code 0)

```
$ docker rm -f lifecycle-demo policy-demo && docker ps -a --format 'table {{.Names}}\t{{.Status}}'
```

```
lifecycle-demo
policy-demo
NAMES     STATUS
dl-web    Up 34 seconds
```

Observation: Both demo containers were deleted. The listing afterwards shows only dl-web (Up 34 seconds), so the removal worked and the image remains.

6.5 Networking Concepts (Steps 27-31)

Concept: Docker's built-in network drivers include bridge (default, single host), host and none. A user-defined bridge network adds DNS-based name resolution between containers and better isolation than the default bridge.

Step 27: List default networks (exit code 0)

```
$ docker network ls
```

```
NETWORK ID     NAME      DRIVER    SCOPE
b33fa5181019   bridge    bridge    local
72b8645dfbf5   host      host      local
678b92f8f1bc   none      null      local
```

Observation: Docker provides three built-in local networks: bridge, host and none (driver null).

Step 28: Create a user-defined bridge network (exit code 0)

```
$ docker network create dl-net
```

```
f8d1ee575f52124fcc6f08736330fcf98f580b241fe1597dd85131ba46db9cf2
```

Observation: The user-defined bridge network dl-net was created; the command returns its ID (f8d1ee575f52...).

Step 29: Start two containers on the network (exit code 0)

```
$ docker run -d --name backend --network dl-net nginx:alpine && docker run -d --name client --network dl-net
  alpine sleep 300
```

```
7e18f3a36e6dd4c0e2a3eaca66447ec17c91ee426332cc3ffbf2dbd5cb2707de
3a1c4faa024cea424d1f6111e2ad23ac25752c464679dc6f5333a06b138ff183

[stderr]
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
ec67817084c6: Download complete
36c38f094eae: Download complete
Digest: sha256:294b683cb724975bec92580e1e685676bd4b50bda910ddb8c51d4cabeaec77e6
Status: Downloaded newer image for alpine:latest
```

Observation: Both containers (backend and client) joined dl-net. alpine:latest was not local, so it was pulled first (digest sha256:294b683c...).

Step 30: Container-to-container communication by name (exit code 0)

```
$ docker exec client sh -c 'ping -c 2 backend && wget -qO- http://backend | head -n 4'
```

```
PING backend (172.18.0.2): 56 data bytes
64 bytes from 172.18.0.2: seq=0 ttl=64 time=0.404 ms
64 bytes from 172.18.0.2: seq=1 ttl=64 time=0.191 ms

--- backend ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.191/0.297/0.404 ms
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
```

Observation: The name 'backend' resolved to 172.18.0.2. Both ping packets arrived (0 % loss, average 0.297 ms) and wget fetched the first four lines of the nginx welcome page, proving container-to-container communication by name.

Step 31: Inspect the network (exit code 0)

```
$ docker network inspect dl-net --format '{{range .Containers}}{{.Name}} -> {{.IPv4Address}}{{println}}{{end}}'
```

```
client -> 172.18.0.3/16
backend -> 172.18.0.2/16
```

Observation: The network contains backend at 172.18.0.2/16 and client at 172.18.0.3/16.

6.6 Storage Concepts (Volumes and Bind Mounts) (Steps 32-36)

Concept: A container's own filesystem is ephemeral. Named volumes (managed by Docker) and bind mounts (host folders) keep data beyond the container lifecycle, for example datasets or model checkpoints.

Step 32: Create a named volume (exit code 0)

```
$ docker volume create dl-data && docker volume ls --filter name=dl-data
```

```
dl-data
DRIVER    VOLUME NAME
local     dl-data
```

Observation: The named volume dl-data was created with the local driver.

Step 33: Write data from one container (exit code 0)

```
$ docker run --rm -v dl-data:/data alpine sh -c "echo 'model checkpoint v1' > /data/checkpoint.txt && ls -l
  /data"
```

```
total 4
-rw-r--r--    1 root     root            20 Oct  6 15:49 checkpoint.txt
```

Observation: A temporary alpine container wrote checkpoint.txt (20 bytes) into the volume and was then deleted by --rm.

Step 34: Read the data from a NEW container (persistence) (exit code 0)

```
$ docker run --rm -v dl-data:/data alpine cat /data/checkpoint.txt
```

```
model checkpoint v1
```

Observation: A different, new container read 'model checkpoint v1' from the volume, which proves that the data outlives the container that wrote it.

Step 35: Bind mount a host folder (exit code 0)

```
$ mkdir -p host_data && echo 'dataset sample from host' > host_data/sample.txt && docker run --rm -v
  "$(pwd)/host_data:/mnt/host" alpine cat /mnt/host/sample.txt
```

```
dataset sample from host
```

Observation: A host folder was bind-mounted into the container, and the container read the file created on the Mac ('dataset sample from host').

Step 36: Inspect the volume (exit code 0)

```
$ docker volume inspect dl-data --format 'Name={{.Name}} Driver={{.Driver}} Mountpoint={{.Mountpoint}}'
```

```
Name=dl-data Driver=local Mountpoint=/var/lib/docker/volumes/dl-data/_data
```

Observation: The volume uses the local driver and lives at /var/lib/docker/volumes/dl-data/_data. This path is inside Docker Desktop's VM, not directly on the macOS file system.

6.7 Cleanup (Step 37)

Step 37: Remove all demo resources (exit code 0)

```
$ docker rm -f dl-web backend client 2>&1; docker network rm dl-net; docker volume rm dl-data; docker ps -a
  --format 'table {{.Names}}\t{{.Status}}'
```

```
dl-web
backend
client
dl-net
dl-data
NAMES     STATUS
```

Observation: The cleanup removed dl-web, backend, client, dl-net and dl-data (each name is echoed back). The final docker ps -a shows only the header row, so no containers remain. The images were kept.

7. Screenshots and Observations

All screenshots were captured on the author's MacBook Air (macOS, Docker Desktop v4.94.0). The seven terminal screenshots are consecutive scroll positions of the single terminal session in which `python3 task9_docker.py` ran; they are shown here in execution order and cropped to the text area, and the step numbers in each heading tell which commands they cover. The three Docker Desktop screenshots show the Containers, Images and Volumes tabs. Each screenshot has its own page.

Notes for transparency.

- Step 37 of the script deletes dl-web, dl-net and dl-data. To have live objects for the Docker Desktop screenshots, `dl-web`, `dl-data` and `dl-net` were re-created by hand after the run (visible at the end of Figure 9). Their IDs (for example 30a02520d1bb) therefore differ from those in the log (562445d82744), and the volume is empty. The images were not deleted.
- The registry name `yourdockerhubuser` in the tag step is a placeholder, so the image appears under that name in Figure 6.

7.1 Installation Check and hello-world (Steps 1-4)

```
(base) prathameshdaulatnagare@prathameshs-MacBook-Air Task9 %    pip3 install reportlab
Collecting reportlab
  Downloading reportlab-5.0.1-py3-none-any.whl.metadata (1.6 kB)
Requirement already satisfied: pillow>=9.0.0 in /opt/anaconda3/lib/python3.13/site-packages (from reportlab) (12.0.0)
Requirement already satisfied: charset-normalizer in /opt/anaconda3/lib/python3.13/site-packages (from reportlab) (3.4.4)
Downloading reportlab-5.0.1-py3-none-any.whl (2.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.0/2.0 MB 41.1 MB/s  0:00:00
Installing collected packages: reportlab
Successfully installed reportlab-5.0.1
(base) prathameshdaulatnagare@prathameshs-MacBook-Air Task9 % python3 task9_docker.py

### [1/37] Docker client/server version
$ docker --version && docker version --format 'Client: {{.Client.Version}} | Server: {{.Server.Version}}'
Docker version 29.8.2, build 7fc2dff
Client: 29.8.2 | Server: 29.8.2
(exit code 0)

### [2/37] Engine information
$ docker info --format 'Containers: {{.Containers}} | Images: {{.Images}} | Storage driver: {{.Driver}} | OS: {{.OperatingSystem}}'
Containers: 0 | Images: 1 | Storage driver: overlayfs | OS: Docker Desktop
(exit code 0)

### [3/37] Run the hello-world test container
$ docker run --rm hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (arm64v8)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
(exit code 0)

### [4/37] Run a detached web container with port mapping
$ docker run -d --name dl-web -p 8080:80 nginx:alpine
562445d82744770349bda1d3365204f0c3badd112b3cce1656a529d2d6b3c68e

[stderr]
Unable to find image 'nginx:alpine' locally
alpine: Pulling from library/nginx
1efec2485f45: Pulling fs layer
a9986cd6f37d: Pulling fs layer
bc862a9910f5: Pulling fs layer
ccea4a153c59: Pulling fs layer
943c389642fa: Pulling fs layer
858f76146c12: Pulling fs layer
766d3311d5df: Pulling fs layer
d766d06974fa: Pulling fs layer
858f76146c12: Download complete
ccea4a153c59: Download complete
```

Figure 1: Terminal session start. After installing reportlab 5.0.1, python3 task9_docker.py runs steps 1 to 3 (versions, engine information, hello-world) and begins step 4, the pull of nginx:alpine.

What the screenshot shows

- Docker client and server both report 29.8.2; the engine has 0 containers and 1 image, storage driver overlayfs.
- hello-world prints its confirmation message and exits with code 0 (step 3).
- Step 4 starts dl-web in the background; because nginx:alpine is not local, the pull progress follows.

7.2 Container Creation, Access, Logs and Resource Usage (Steps 4-9)

```
bc862a9910f5: Pulling fs layer
ccea4a153c59: Pulling fs layer
943c389642fa: Pulling fs layer
858f76146c12: Pulling fs layer
766d3311d5df: Pulling fs layer
d766d06974fa: Pulling fs layer
858f76146c12: Download complete
ccea4a153c59: Download complete
bc862a9910f5: Download complete
d766d06974fa: Download complete
766d3311d5df: Download complete
a9986cd6f37d: Download complete
a9986cd6f37d: Pull complete
858f76146c12: Pull complete
ccea4a153c59: Pull complete
766d3311d5df: Pull complete
943c389642fa: Download complete
bc862a9910f5: Pull complete
d766d06974fa: Pull complete
943c389642fa: Pull complete
dffc70821449: Download complete
6c2852a60c45: Download complete
1efec2485f45: Download complete
1efec2485f45: Pull complete
Digest: sha256:df221db836e1754089190208cee7eeda94f233197056426eda74a43ab1abeac2
Status: Downloaded newer image for nginx:alpine
(exit code 0)

### [5/37] List running containers
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED                  STATUS                  PORTS                                       NAMES
562445d82744   nginx:alpine   "/docker-entrypoint.…"   Less than a second ago   Up Less than a second   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   dl-web
(exit code 0)

### [6/37] Call the container from the host
$ curl -s -o /dev/null -w 'HTTP status: %{http_code}\n' http://localhost:8080
HTTP status: 200
(exit code 0)

### [7/37] Run a command inside the running container
$ docker exec dl-web sh -c 'echo Hello from inside the container && nginx -v && hostname'
Hello from inside the container
562445d82744

[stderr]
nginx version: nginx/1.31.6
(exit code 0)

### [8/37] View container logs
$ docker logs --tail 5 dl-web
192.168.65.1 - - [06/Oct/2026:15:48:29 +0000] "GET / HTTP/1.1" 200 896 "-" "curl/8.16.0" "-"

[stderr]
2026/10/06 15:48:29 [notice] 1#1: start worker process 36
2026/10/06 15:48:29 [notice] 1#1: start worker process 37
2026/10/06 15:48:29 [notice] 1#1: start worker process 38
2026/10/06 15:48:29 [notice] 1#1: start worker process 39
(exit code 0)

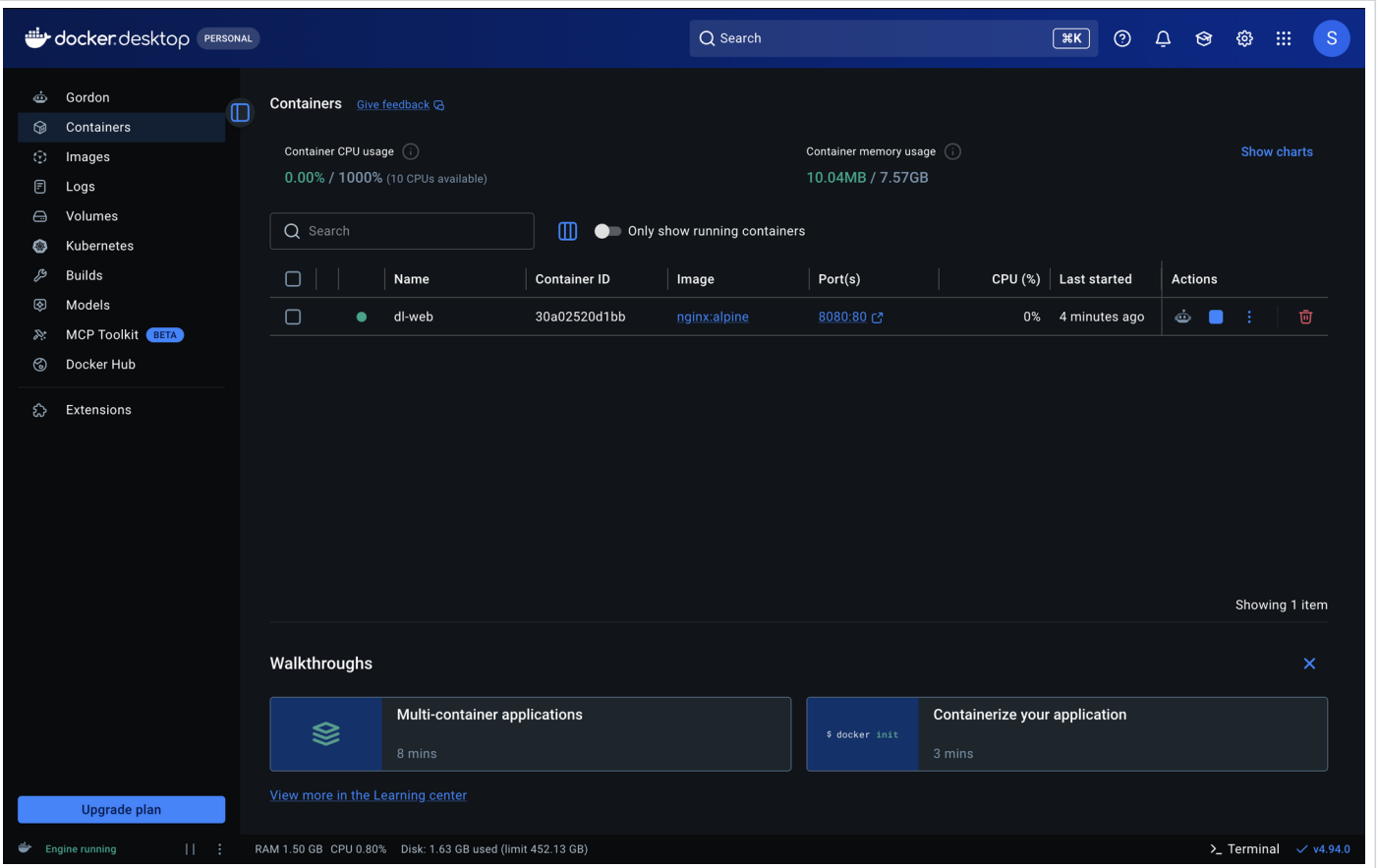
### [9/37] Resource usage snapshot
$ docker stats --no-stream dl-web
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT     MEM %     NET I/O         BLOCK I/O     PIDS
562445d82744   dl-web    0.00%     10.25MiB / 7.748GiB   0.13%     1.63kB / 1.6kB   0B / 8.19kB   11
(exit code 0)
```

Figure 2: Completion of the nginx pull and steps 5 to 9: docker ps, curl to the published port, docker exec, docker logs and docker stats.

What the screenshot shows

- Pull finished with digest sha256:df221db8... and status 'Downloaded newer image for nginx:alpine'.
- docker ps shows dl-web with 0.0.0.0:8080->80/tcp; curl returns HTTP status 200.
- docker exec prints the container hostname 562445d82744; the log shows the curl request (GET / 200).
- docker stats: 0.00 % CPU, 10.25 MiB of 7.748 GiB memory, 11 PIDs.

7.3 Docker Desktop - Containers Tab



Enlarged detail of the same screen

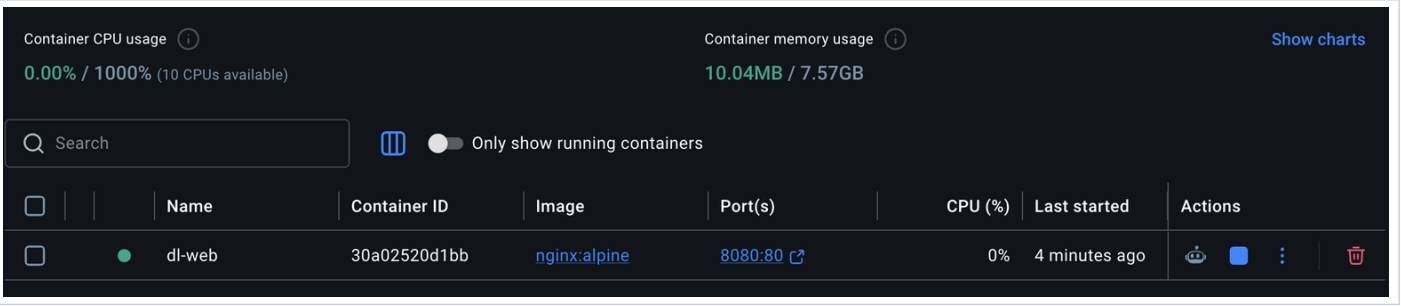


Figure 3: Docker Desktop, Containers tab, with dl-web (nginx:alpine) running on 8080:80. Engine status 'Engine running', Docker Desktop v4.94.0. The lower image enlarges the table area of the same screen.

What the screenshot shows

- Row for dl-web: container ID 30a02520d1bb, image nginx:alpine, port 8080:80, CPU 0 %, started 4 minutes ago.
- Summary at the top: container CPU 0.00 % of 1000 % (10 CPUs available) and memory 10.04 MB of 7.57 GB.
- The ID differs from the log (562445d82744) because the container was re-created by hand after the scripted cleanup (see the note at the start of Section 7).

7.4 Failed Inspect, Image Pull, Registry Search, Image List and History (Steps 10-14)

```
(exit code 0)

### [10/37] Inspect container metadata
$ docker inspect --format 'Name={{.Name}} State={{.State.Status}} IP={{.NetworkSettings.IPAddress}} Image={{.Config.Image}}' dl-web


[stderr]
template parsing error: template: :1:60: executing "" at <.NetworkSettings.IPAddress>: map has no entry for key "IPAddress"
(exit code 1)

### [11/37] Pull an image from the registry (Docker Hub)
$ docker pull python:3.11-slim
3.11-slim: Pulling from library/python
7ca0c331ecfd: Pulling fs layer
9eb0fb93ee08: Pulling fs layer
bbeda6b4abb7: Pulling fs layer
73a63ac3ec4a: Pulling fs layer
9eb0fb93ee08: Download complete
7ca0c331ecfd: Download complete
58b126dd1754: Download complete
56f7c999317d: Download complete
bbeda6b4abb7: Download complete
73a63ac3ec4a: Download complete
bbeda6b4abb7: Pull complete
9eb0fb93ee08: Pull complete
7ca0c331ecfd: Pull complete
73a63ac3ec4a: Pull complete
Digest: sha256:0dd364ba7e10242f07755449e3a3d0e35f9efd987952737b90def6709ab0c5ce
Status: Downloaded newer image for python:3.11-slim
docker.io/library/python:3.11-slim
(exit code 0)

### [12/37] Search the registry
$ docker search tensorflow --limit 5
NAME                    DESCRIPTION                                     STARS     OFFICIAL
tensorflow/tensorflow   Official Docker images for the machine learn…   2932
rocm/tensorflow         Tensorflow with ROCm backend support            86
photoprism/tensorflow   TensorFlow for C Build Environment              0
corpusops/tensorflow    https://github.com/corpusops/docker-tensorfl…   0
rootpublic/tensorflow                                                   0
(exit code 0)

### [13/37] List local images
$ docker images
IMAGE               ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest  5e2309035332       17.1kB         4.77kB
nginx:alpine        df221db836e1       93.9MB         27.1MB   U
python:3.11-slim    0dd364ba7e10        215MB         48.1MB
(exit code 0)

### [14/37] Image layers (history)
$ docker history python:3.11-slim
IMAGE          CREATED        CREATED BY                                      SIZE      COMMENT
0dd364ba7e10   14 hours ago   CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      14 hours ago   RUN /bin/sh -c set -eux;  for src in idle3 p…   16.4kB    buildkit.dockerfile.v0
<missing>      14 hours ago   RUN /bin/sh -c set -eux;   savedAptMark="$(a…   52.1MB    buildkit.dockerfile.v0
<missing>      14 hours ago   ENV PYTHON_SHA256=bfb74ad39efae27cda510f134a…   0B        buildkit.dockerfile.v0
<missing>      14 hours ago   ENV PYTHON_VERSION=3.11.17                      0B        buildkit.dockerfile.v0
<missing>      14 hours ago   ENV GPG_KEY=A035C8C19219BA821ECEA86B64E628F8…   0B        buildkit.dockerfile.v0
<missing>      14 hours ago   RUN /bin/sh -c set -eux;  apt-get update;  a…   4.99MB    buildkit.dockerfile.v0
<missing>      14 hours ago   ENV LANG=C.UTF-8                                0B        buildkit.dockerfile.v0
<missing>      14 hours ago   ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buildkit.dockerfile.v0
<missing>      40 hours ago   # debian.sh --arch 'arm64' out/ 'trixie' '@1…   110MB     debuerreotype 0.17
(exit code 0)
```

Figure 4: Step 10 (docker inspect template error), step 11 (pull python:3.11-slim), step 12 (docker search tensorflow), step 13 (docker images) and the beginning of step 14 (docker history).

What the screenshot shows

- Step 10 is the only failed command: 'map has no entry for key IPAddress', exit code 1 (diagnosis in Section 4.7).
- python:3.11-slim is pulled as 4 layers; the search lists tensorflow/tensorflow with 2932 stars first.
- docker images lists hello-world, nginx:alpine and python:3.11-slim with sizes 17.1 kB, 93.9 MB and 215 MB.

7.5 Layer History and Building the Custom Image (Steps 14-15)

```
<missing>      14 hours ago   ENV PYTHON_VERSION=3.11.17                   0B        buildkit.dockerfile.v0
<missing>      14 hours ago   ENV GPG_KEY=A035C8C19219BA821ECEA86B64E628F8…   0B        buildkit.dockerfile.v0
<missing>      14 hours ago   RUN /bin/sh -c set -eux;  apt-get update;  a…   4.99MB    buildkit.dockerfile.v0
<missing>      14 hours ago   ENV LANG=C.UTF-8                             0B        buildkit.dockerfile.v0
<missing>      14 hours ago   ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buildkit.dockerfile.v0
<missing>      40 hours ago   # debian.sh --arch 'arm64' out/ 'trixie' '@1…   110MB     debuerreotype 0.17
(exit code 0)

### [15/37] Build a custom DL-style image from a Dockerfile
$ docker build -t dl-app:1.0 ./dl_app

[stderr]
#0 building with "desktop-linux" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 148B done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/python:3.11-slim
#2 DONE 0.0s

#3 [internal] load .dockerignore
#3 transferring context: 2B done
#3 DONE 0.0s

#4 [internal] load build context
#4 transferring context: 674B done
#4 DONE 0.0s

#5 [1/4] FROM docker.io/library/python:3.11-slim@sha256:0dd364ba7e10242f07755449e3a3d0e35f9efd987952737b90def6709ab0c5ce
#5 resolve docker.io/library/python:3.11-slim@sha256:0dd364ba7e10242f07755449e3a3d0e35f9efd987952737b90def6709ab0c5ce 0.0s done
#5 extracting sha256:bbeda6b4abb749f743f4547ef5a28070a14de8caef211daa5afb78d5b5fa21ba
#5 extracting sha256:bbeda6b4abb749f743f4547ef5a28070a14de8caef211daa5afb78d5b5fa21ba 1.4s done
#5 DONE 1.4s

#5 [1/4] FROM docker.io/library/python:3.11-slim@sha256:0dd364ba7e10242f07755449e3a3d0e35f9efd987952737b90def6709ab0c5ce
#5 extracting sha256:9eb0fb93ee0808559f67707773b41df45d2dd34a5b4aec77d2f7a6bc36d05a51 0.1s done
#5 extracting sha256:73a63ac3ec4a2ac51ab19307c2a807cd83546a50407c2c52470ab27a240d648d
#5 extracting sha256:73a63ac3ec4a2ac51ab19307c2a807cd83546a50407c2c52470ab27a240d648d 0.5s done
#5 DONE 2.0s

#5 [1/4] FROM docker.io/library/python:3.11-slim@sha256:0dd364ba7e10242f07755449e3a3d0e35f9efd987952737b90def6709ab0c5ce
#5 extracting sha256:7ca0c331ecfd0200dc735b8a2ac2be32f9c43d5ea0945cbbaf98c04335fdeeb5 0.0s done
#5 DONE 2.0s

#6 [2/4] WORKDIR /app
#6 DONE 0.9s

#7 [3/4] RUN pip install --no-cache-dir numpy
#7 1.900 Collecting numpy
#7 2.016   Downloading numpy-2.4.6-cp311-cp311-manylinux_2_27_aarch64.manylinux_2_28_aarch64.whl.metadata (6.6 kB)
#7 2.050 Downloading numpy-2.4.6-cp311-cp311-manylinux_2_27_aarch64.manylinux_2_28_aarch64.whl (16.0 MB)
#7 5.481    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 16.0/16.0 MB 4.2 MB/s eta 0:00:00
#7 5.557 Installing collected packages: numpy
#7 7.350 Successfully installed numpy-2.4.6
#7 7.350 WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: https
/pip.pypa.io/warnings/venv
#7 7.442
#7 7.442 [notice] A new release of pip is available: 24.0 -> 26.2.1
#7 7.442 [notice] To update, run: pip install --upgrade pip
#7 DONE 7.6s

#8 [4/4] COPY app.py .
#8 DONE 0.0s
```

Figure 5: End of the python:3.11-slim layer history and the BuildKit output of docker build -t dl-app:1.0 ./dl_app.

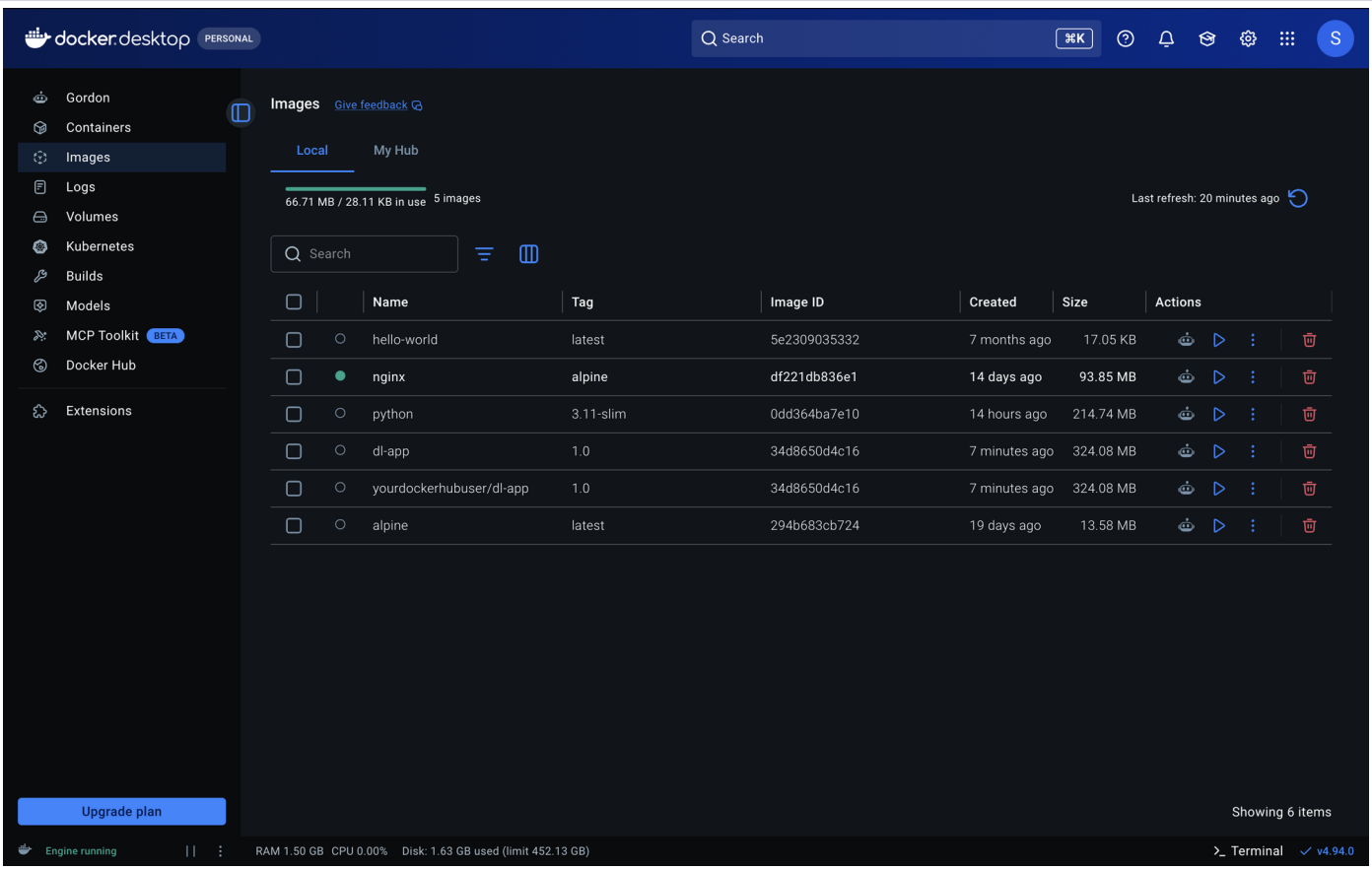
What the screenshot shows

- The history ends with the 110 MB Debian arm64 base layer; the ENV and CMD layers are 0 B.
- Build stages: [1/4] FROM python:3.11-slim, [2/4] WORKDIR /app, [3/4] RUN pip install numpy (7.6 s, NumPy 2.4.6), [4/4] COPY app.py.
- The build context was only 674 B; the pip warnings about running as root and a new pip release do not fail the build.

Steps covered by this screenshot

Step	Command	Exit	Result
14	`docker history python:3.11-slim`	0	The history lists the layers of python:3.11-slim from CMD ["python3"] down to the 110 MB Debian (arm64) base layer
15	`docker build -t dl-app:1.0 ./dl_app`	0	The four build steps ran in order: FROM, WORKDIR /app, RUN pip install numpy (7.6 s, NumPy 2.4.6 aarch64 wheel of 16.0 MB) and COPY app.py

7.6 Docker Desktop - Images Tab



Enlarged detail of the same screen

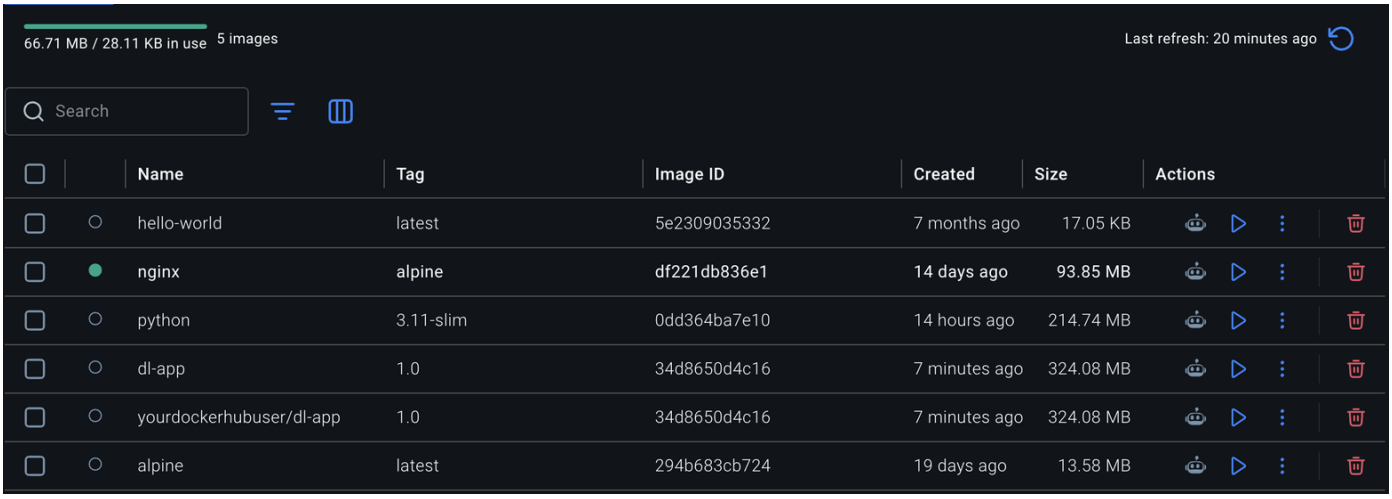


Figure 6: Docker Desktop, Images tab, listing the six local image entries. The lower image enlarges the table area of the same screen.

What the screenshot shows

- hello-world:latest (17.05 KB), nginx:alpine (93.85 MB, green dot = in use), python:3.11-slim (214.74 MB), alpine:latest (13.58 MB).
- dl-app:1.0 and yourdockerhubuser/dl-app:1.0 show the same image ID 34d8650d4c16 and the same size 324.08 MB: two tags, one image.
- The sizes match the CLI listing of steps 13 and 17.

7.7 Running the DL App, Tagging and Lifecycle from Create to Stop (Steps 15-22)

```
#9 exporting to image
#9 exporting layers
#9 exporting layers 2.4s done
#9 exporting manifest sha256:f199e40ff0715e54c75d43014eaeca496644a3e7d68c7754b8c76824884de3d5 done
#9 exporting config sha256:22c9baabe670a037fced1dbf13b8b48ea2675e323c2ef9e04d86268989e73f6d done
#9 exporting attestation manifest sha256:27bea9899e1c00ca8993bf970a9a47a3c07deecbc5275774f44bc81e31312600 done
#9 exporting manifest list sha256:34d8650d4c166ed015da916d2feeb849735741166386ba77f644b8c7dec5489d done
#9 naming to docker.io/library/dl-app:1.0 done
#9 unpacking to docker.io/library/dl-app:1.0
#9 unpacking to docker.io/library/dl-app:1.0 0.5s done
#9 DONE 2.9s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/lx36dzav3rthmuxqjcs3mxmrp
(exit code 0)

### [16/37] Run the custom image
$ docker run --rm dl-app:1.0
Containerized DL inference app
NumPy version : 2.4.6
Output shape  : (4, 3)
Predicted classes: [2, 0, 0, 0]
(exit code 0)

### [17/37] Tag the image for a registry
$ docker tag dl-app:1.0 yourdockerhubuser/dl-app:1.0 && docker images yourdockerhubuser/dl-app
IMAGE                          ID             DISK USAGE   CONTENT SIZE   EXTRA
yourdockerhubuser/dl-app:1.0   34d8650d4c16        324MB         70.7MB
(exit code 0)

### [18/37] Create a container without starting it
$ docker create --name lifecycle-demo python:3.11-slim sleep 600 && docker ps -a --filter name=lifecycle-demo --format 'table {{.Names}}\t{{.Status}}'
3817f42343b9146ec3087eb9de5ca0e848c11f3008808ccb3c2364487f8730b2
NAMES            STATUS
lifecycle-demo   Created
(exit code 0)

### [19/37] Start
$ docker start lifecycle-demo && docker ps --filter name=lifecycle-demo --format 'table {{.Names}}\t{{.Status}}'
lifecycle-demo
NAMES            STATUS
lifecycle-demo   Up Less than a second
(exit code 0)

### [20/37] Pause
$ docker pause lifecycle-demo && docker ps --filter name=lifecycle-demo --format 'table {{.Names}}\t{{.Status}}'
lifecycle-demo
NAMES            STATUS
lifecycle-demo   Up Less than a second (Paused)
(exit code 0)

### [21/37] Unpause
$ docker unpause lifecycle-demo && docker ps --filter name=lifecycle-demo --format 'table {{.Names}}\t{{.Status}}'
lifecycle-demo
NAMES            STATUS
lifecycle-demo   Up Less than a second
(exit code 0)

### [22/37] Stop (graceful, SIGTERM then SIGKILL)
$ docker stop -t 2 lifecycle-demo && docker ps -a --filter name=lifecycle-demo --format 'table {{.Names}}\t{{.Status}}'
lifecycle-demo
NAMES            STATUS
lifecycle-demo   Exited (137) Less than a second ago
(exit code 0)
```

Figure 7: Build export, docker run of dl-app:1.0, docker tag, and the lifecycle commands create, start, pause, unpause and stop with the container status after each.

What the screenshot shows

- The container prints 'Containerized DL inference app', NumPy 2.4.6, output shape (4, 3) and predicted classes [2, 0, 0, 0].
- Status progression: Created, Up, Up (Paused), Up, Exited (137) Less than a second ago.
- The tag step lists yourdockerhubuser/dl-app:1.0 with ID 34d8650d4c16 (324 MB disk usage, 70.7 MB content).

7.8 Restart, Kill, Restart Policy, Removal and Networks (Steps 23-30)

```
lifecycle-demo   Exited (137) Less than a second ago
(exit code 0)

### [23/37] Restart
$ docker restart lifecycle-demo && docker ps --filter name=lifecycle-demo --format 'table {{.Names}}\t{{.Status}}'
lifecycle-demo
NAMES           STATUS
lifecycle-demo   Up Less than a second
(exit code 0)

### [24/37] Kill (immediate SIGKILL)
$ docker kill lifecycle-demo && docker ps -a --filter name=lifecycle-demo --format 'table {{.Names}}\t{{.Status}}'
lifecycle-demo
NAMES           STATUS
lifecycle-demo   Exited (137) Less than a second ago
(exit code 0)

### [25/37] Restart policy demo
$ docker run -d --name policy-demo --restart unless-stopped nginx:alpine && docker inspect --format 'RestartPolicy={{.HostConfig.RestartPolicy.Name}}' policy-demo
9820d01540a0ea5504cc38c63862186f68536cbdae4bfcb95d41df3f64ad575b
RestartPolicy=unless-stopped
(exit code 0)

### [26/37] Remove containers
$ docker rm -f lifecycle-demo policy-demo && docker ps -a --format 'table {{.Names}}\t{{.Status}}'
lifecycle-demo
policy-demo
NAMES     STATUS
dl-web    Up 34 seconds
(exit code 0)

### [27/37] List default networks
$ docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
b33fa5181019   bridge    bridge    local
72b8645dfbf5   host      host      local
678b92f8f1bc   none      null      local
(exit code 0)

### [28/37] Create a user-defined bridge network
$ docker network create dl-net
f8d1ee575f52124fcc6f08736330fcf98f580b241fe1597dd85131ba46db9cf2
(exit code 0)

### [29/37] Start two containers on the network
$ docker run -d --name backend --network dl-net nginx:alpine && docker run -d --name client --network dl-net alpine sleep 300
7e18f3a36e6dd4c0e2a3eaca66447ec17c91ee426332cc3ffbf2dbd5cb2707de
3a1c4faa024cea424d1f6111e2ad23ac25752c464679dc6f5333a06b138ff183

[stderr]
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
ec67817084c6: Download complete
36c38f094eae: Download complete
Digest: sha256:294b683cb724975bec92580e1e685676bd4b50bda910ddb8c51d4cabeaec77e6
Status: Downloaded newer image for alpine:latest
(exit code 0)

### [30/37] Container-to-container communication by name
$ docker exec client sh -c 'ping -c 2 backend && wget -qO- http://backend | head -n 4'
PING backend (172.18.0.2): 56 data bytes
64 bytes from 172.18.0.2: seq=0 ttl=64 time=0.404 ms
64 bytes from 172.18.0.2: seq=1 ttl=64 time=0.191 ms
```

Figure 8: Steps 23 to 30: restart, kill, restart policy, container removal, default networks, creation of dl-net, two containers on dl-net and the beginning of the name-resolution test.

What the screenshot shows

- restart returns Up; kill ends in Exited (137); RestartPolicy=unless-stopped is confirmed by docker inspect.
- After docker rm -f only dl-web remains (Up 34 seconds). The default networks are bridge, host and none.
- alpine:latest is pulled when the client container starts; the ping to 'backend' resolves to 172.18.0.2.

7.9 Network Inspection, Volumes, Bind Mount and Cleanup (Steps 30-37)

```
64 bytes from 172.18.0.2: seq=1 ttl=64 time=0.191 ms

--- backend ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.191/0.297/0.404 ms
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
(exit code 0)

### [31/37] Inspect the network
$ docker network inspect dl-net --format '{{range .Containers}}{{.Name}} -> {{.IPv4Address}}{{println}}{{end}}'
client -> 172.18.0.3/16
backend -> 172.18.0.2/16
(exit code 0)

### [32/37] Create a named volume
$ docker volume create dl-data && docker volume ls --filter name=dl-data
dl-data
DRIVER    VOLUME NAME
local     dl-data
(exit code 0)

### [33/37] Write data from one container
$ docker run --rm -v dl-data:/data alpine sh -c "echo 'model checkpoint v1' > /data/checkpoint.txt && ls -l /data"
total 4
-rw-r--r--    1 root     root            20 Oct  6 15:49 checkpoint.txt
(exit code 0)

### [34/37] Read the data from a NEW container (persistence)
$ docker run --rm -v dl-data:/data alpine cat /data/checkpoint.txt
model checkpoint v1
(exit code 0)

### [35/37] Bind mount a host folder
$ mkdir -p host_data && echo 'dataset sample from host' > host_data/sample.txt && docker run --rm -v "$(pwd)/host_data:/mnt/host" alpine cat /mnt/host/sample.txt
dataset sample from host
(exit code 0)

### [36/37] Inspect the volume
$ docker volume inspect dl-data --format 'Name={{.Name}} Driver={{.Driver}} Mountpoint={{.Mountpoint}}'
Name=dl-data Driver=local Mountpoint=/var/lib/docker/volumes/dl-data/_data
(exit code 0)

### [37/37] Remove all demo resources
$ docker rm -f dl-web backend client 2>&1; docker network rm dl-net; docker volume rm dl-data; docker ps -a --format 'table {{.Names}}\t{{.Status}}'
dl-web
backend
client
dl-net
dl-data
NAMES     STATUS
(exit code 0)

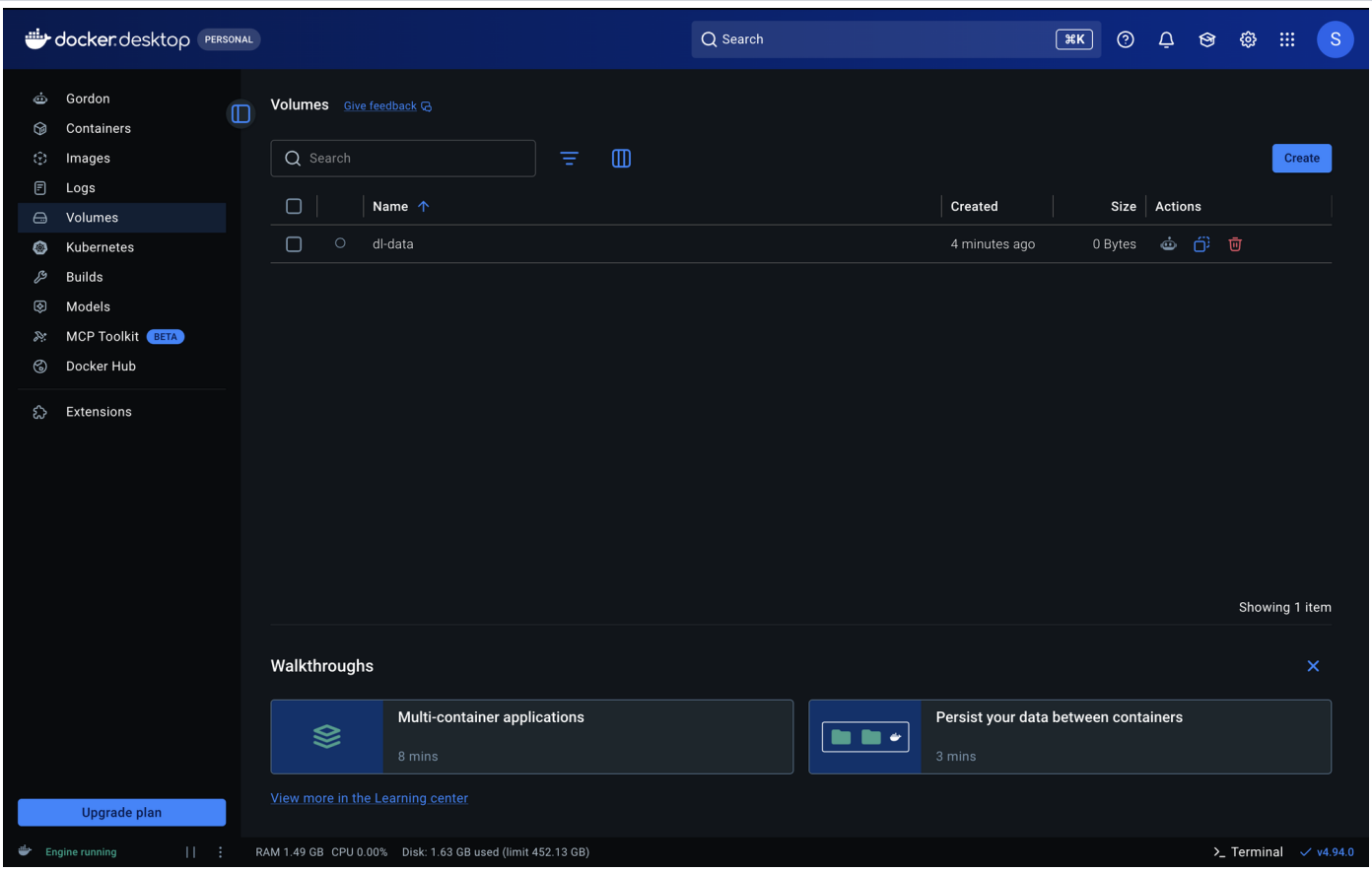
PDF written: Task9_Docker_Report.pdf
Log saved: docker_command_log.txt
(base) prathameshdaulatnagare@prathameshs-MacBook-Air Task9 % docker run -d --name dl-web -p 8080:80 nginx:alpine
docker volume create dl-data
docker network create dl-net
30a02520d1bbd3d260a7a38d39e8242bd55cbbd5a27dc90094da501bfee761b4
dl-data
31740b6bbb7dc87f738ecf683eb6f3fa40820cdad2d29c0708663e964d627ab9
```

Figure 9: End of the ping and wget test, network inspection, volume write/read, bind mount, volume inspection and cleanup. The last lines show the script finishing and the three resources re-created by hand for the GUI screenshots.

What the screenshot shows

- Ping: 2 packets sent and received, 0 % loss, average 0.297 ms; wget returns the nginx welcome page.
- dl-net contains client 172.18.0.3/16 and backend 172.18.0.2/16.
- The file written to dl-data by one container is read by another ('model checkpoint v1'); the bind mount returns 'dataset sample from host'.
- The script ends with 'PDF written' and 'Log saved'; afterwards docker run -d --name dl-web -p 8080:80 nginx:alpine, docker volume create dl-data and docker network create dl-net were typed manually.

7.10 Docker Desktop - Volumes Tab



Enlarged detail of the same screen

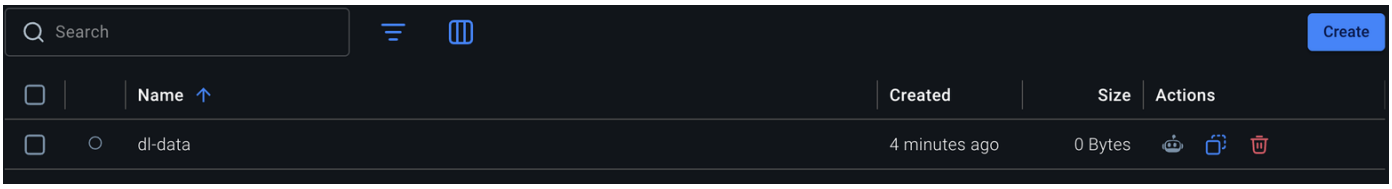


Figure 10: Docker Desktop, Volumes tab, showing the named volume dl-data. The lower image enlarges the table area of the same screen.

What the screenshot shows

- dl-data was created 4 minutes before the screenshot and is 0 Bytes because it was re-created empty after the scripted cleanup (the file written in step 33 was deleted with the original volume in step 37).
- The Volumes tab offers the same operations as docker volume ls, inspect and rm.

8. Results

The table summarises the recorded run. Values are taken from the command outputs in Section 6 and the Docker Desktop screenshots.

Metric	Value
Commands executed / succeeded	37 / 36 (97.3 %); the one failure is step 10
Images pulled from Docker Hub	nginx:alpine (8 layers), python:3.11-slim (4 layers), alpine:latest; hello-world was already local
Image built	dl-app:1.0, ID 34d8650d4c16, 324 MB disk usage, 70.7 MB content size, build export 2.9 s
DL application output	NumPy 2.4.6, output shape (4, 3), predicted classes [2, 0, 0, 0]
Published port test	host port 8080 to container port 80, HTTP status 200
Idle resource use of dl-web	CPU 0.00 %, memory 10.25 MiB of 7.748 GiB (0.13 %), 11 PIDs
Container-to-container ping	2 of 2 packets received, 0 % loss, min/avg/max 0.191 / 0.297 / 0.404 ms
Volume persistence	checkpoint.txt (20 bytes) written by one container and read by a second one
Cleanup result	dl-web, backend, client, dl-net and dl-data removed; docker ps -a empty

8.1 Results by approach area

Approach area	Steps	Succeeded
Install Docker and Verify the Setup	1-3	3/3
Create and Manage Containers	4-10	6/7
Explore Docker Images and Registries	11-17	7/7
Container Lifecycle Operations	18-26	9/9
Networking Concepts	27-31	5/5
Storage Concepts (Volumes and Bind Mounts)	32-36	5/5
Cleanup	37	1/1

8.2 Lifecycle states observed (steps 18-26)

Step	Command	Status reported by docker ps
18	`docker create`	Created
19	`docker start`	Up Less than a second
20	`docker pause`	Up Less than a second (Paused)
21	`docker unpause`	Up Less than a second
22	`docker stop -t 2`	Exited (137) Less than a second ago
23	`docker restart`	Up Less than a second
24	`docker kill`	Exited (137) Less than a second ago
26	`docker rm -f`	container no longer listed (only dl-web remains, Up 34 seconds)

8.3 Image inventory (steps 13 and 17, Figure 6)

Image	Image ID	Disk usage (CLI) / Docker Desktop size
hello-world:latest	`5e2309035332`	17.1 kB / 17.05 KB
nginx:alpine	`df221db836e1`	93.9 MB / 93.85 MB
python:3.11-slim	`0dd364ba7e10`	215 MB / 214.74 MB
alpine:latest	`294b683cb724`	(listed in Figure 6) 13.58 MB
dl-app:1.0 and yourdockerhubuser/dl-app:1.0	`34d8650d4c16`	324 MB / 324.08 MB

8.4 Network and storage objects

Object	Details from the log
Default networks	bridge (b33fa5181019), host (72b8645dfbf5), none (678b92f8f1bc), all local scope
dl-net (bridge)	ID f8d1ee575f52...; backend 172.18.0.2/16, client 172.18.0.3/16
Volume dl-data	driver local, mountpoint /var/lib/docker/volumes/dl-data/_data, holds checkpoint.txt = 'model checkpoint v1'
Bind mount	host_data/ on the Mac mounted at /mnt/host; file sample.txt = 'dataset sample from host'

9. Observations

- **Architecture.** Client and server versions are identical (29.8.2) and all CLI commands were answered by the engine inside Docker Desktop; on the Mac the engine and its paths (for example /var/lib/docker/volumes) live in a Linux VM.
- **Images are layered and shared.** nginx:alpine was pulled as 8 layers and python:3.11-slim as 4 layers; the history of python:3.11-slim shows that metadata instructions are 0 B while the Debian base layer is 110 MB. dl-app:1.0 has 324 MB disk usage against 215 MB for its base image, the difference being NumPy and the app.
- **Tags are only names.** dl-app:1.0 and yourdockerhubuser/dl-app:1.0 show the same ID (34d8650d4c16) and size in both the CLI and Docker Desktop.
- **Containers are lightweight.** The idle nginx container used 10.25 MiB of memory and 11 processes, and it answered through the published port with HTTP 200.
- **The lifecycle commands behave as documented.** Each transition (Created, Running, Paused, Running, Exited, Running, Exited) was confirmed in the status column. Both the 2-second stop and the kill ended in code 137, because the sleep process did not react to SIGTERM before the timeout.
- **Name-based networking works on user-defined bridges.** The client reached backend by name with 0 % packet loss and an average round-trip of 0.297 ms, and received the nginx welcome page.
- **Container filesystems are temporary, volumes are not.** A file written by a container that was then deleted could still be read by a new container through the named volume, and a bind mount exposed a Mac folder inside a container.
- **One command failed and was understood.** The inspect template in step 10 referenced a field that does not exist in this engine's output; the same information was obtained from a different command, which shows that docker inspect output formats depend on the Docker version.
- **The GUI mirrors the CLI.** The Containers, Images and Volumes tabs in Figures 3, 6 and 10 show the same names, IDs and sizes as the command output.

10. Known Limitations

- **The model is a demonstration.** dl_app/app.py uses random weights with a fixed seed; it shows how to package and run a deep-learning workload, not model accuracy. No GPU was used (Docker Desktop on a Mac has no NVIDIA GPU access).
- **Architecture-specific results.** The images were pulled for arm64 (Apple Silicon). Sizes, digests and IDs will differ on other machines, and moving tags such as python:3.11-slim change over time.
- **Unpinned dependency.** The Dockerfile installs the latest NumPy at build time (2.4.6 here), and the image runs as root. For reproducible production images the version should be pinned and a non-root user added.
- **Registry upload not performed.** Registries were explored through pull, search and tag; no Docker Hub login or docker push was made, and the tag uses a placeholder account name.
- **Single-host scope.** Only bridge networking on one host was exercised; host, overlay and macvlan networks, Docker Compose and orchestration were outside this task.
- **One step failed.** Step 10 (docker inspect with an unsupported template field) is recorded as it happened; the suggested replacement was not re-run.
- **Screenshots taken after cleanup.** The Docker Desktop screenshots show re-created resources, so their IDs differ from the log and the volume is empty (see Section 7).

11. Conclusion

This task covered the full path from installation to cleanup: Docker Desktop and the CLI were verified (client and server 29.8.2), an nginx container was created, reached through a published port, inspected and measured, and a custom image for a NumPy deep-learning inference script was built from a Dockerfile, run and tagged. The complete lifecycle (create, start, pause, unpause, stop, restart, kill, remove) was exercised with the status confirmed after every command, two containers communicated by name over a user-defined bridge network, and a named volume and a bind mount demonstrated persistent storage. 36 of 37 commands completed successfully, and the single failure (a version-dependent inspect template) was diagnosed without affecting the rest of the work. Together with the command log, the ten screenshots and the observations above, the report documents how Docker packages applications with their dependencies into portable images, runs them as isolated containers, connects them through networks and keeps their data in volumes.